Introduction to Machine Learning

Lecture 10: Support Vector Machine

Ali Bereyhi

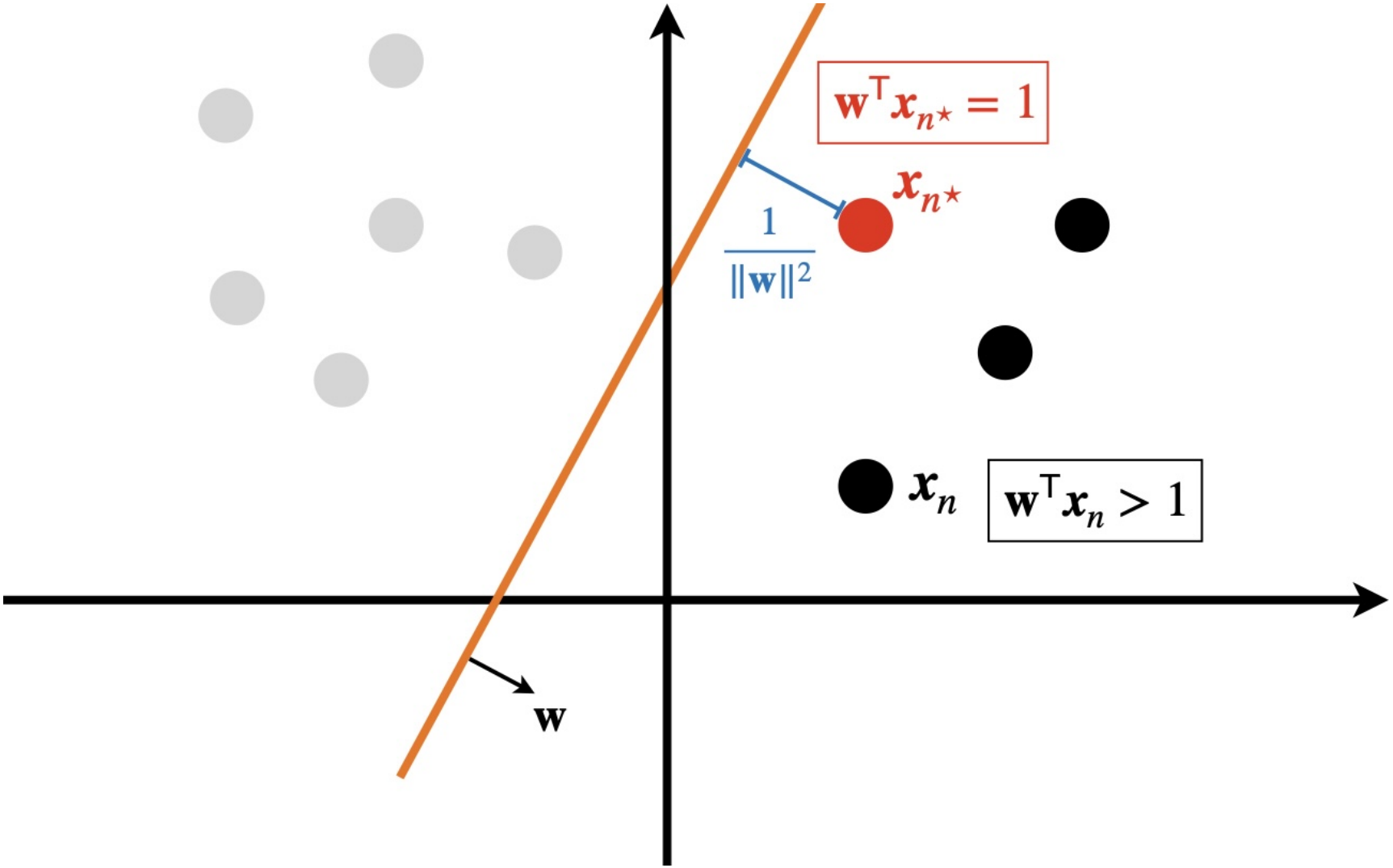
ali.bereyhi@utoronto.ca

Department of Electrical and Computer Engineering

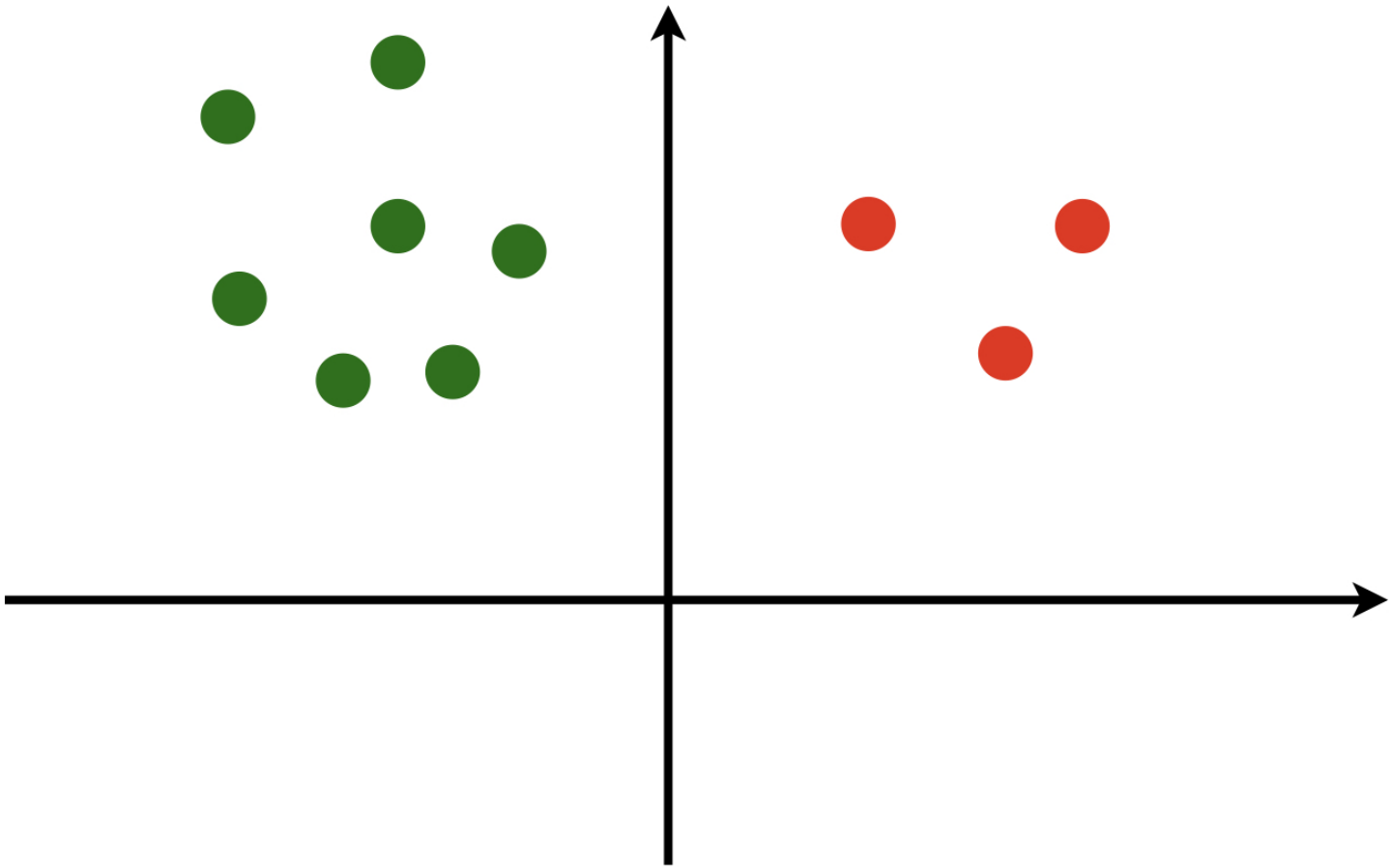
University of Toronto

Winter 2026

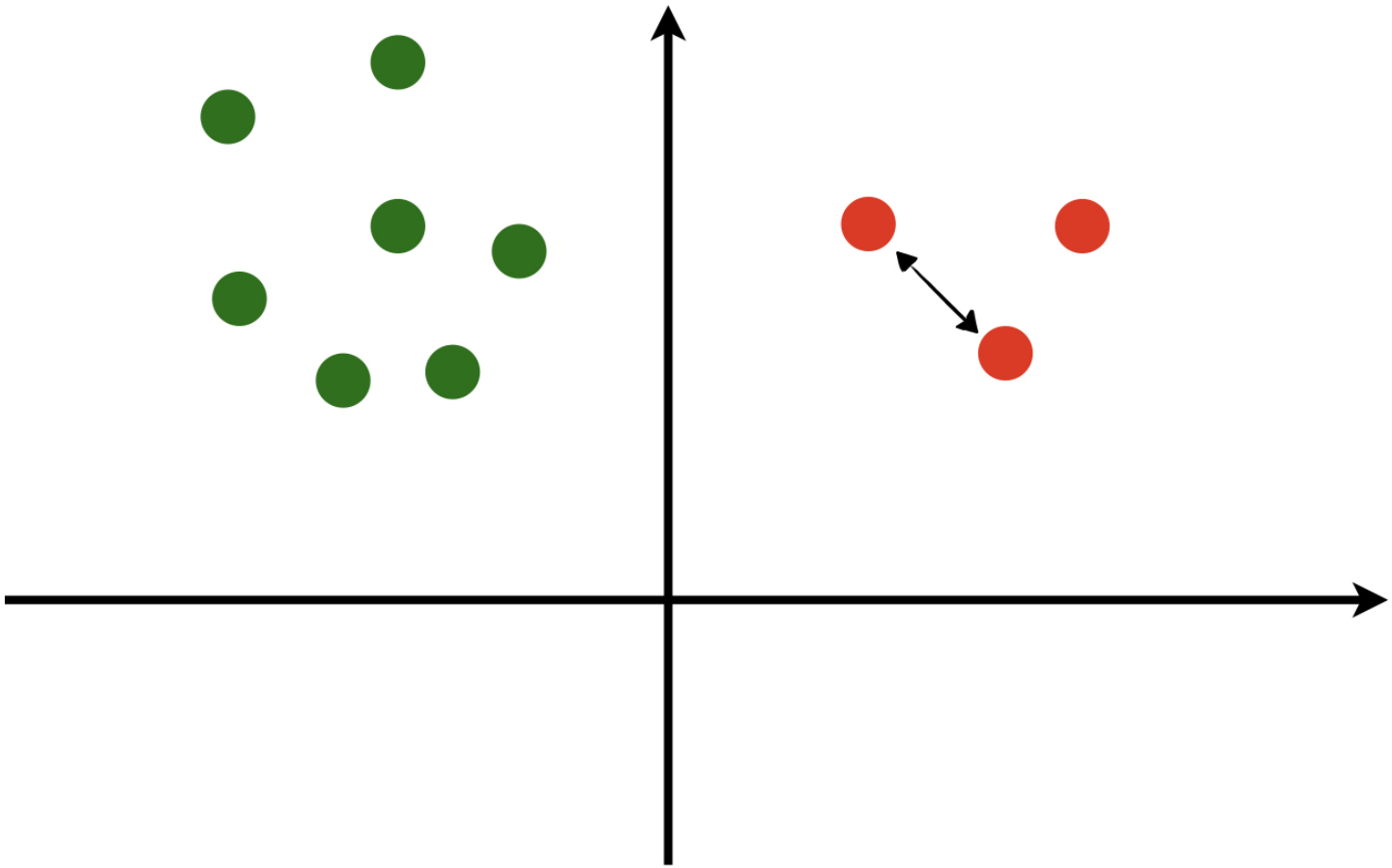
Quick Recap: *Support Vector Classifier*



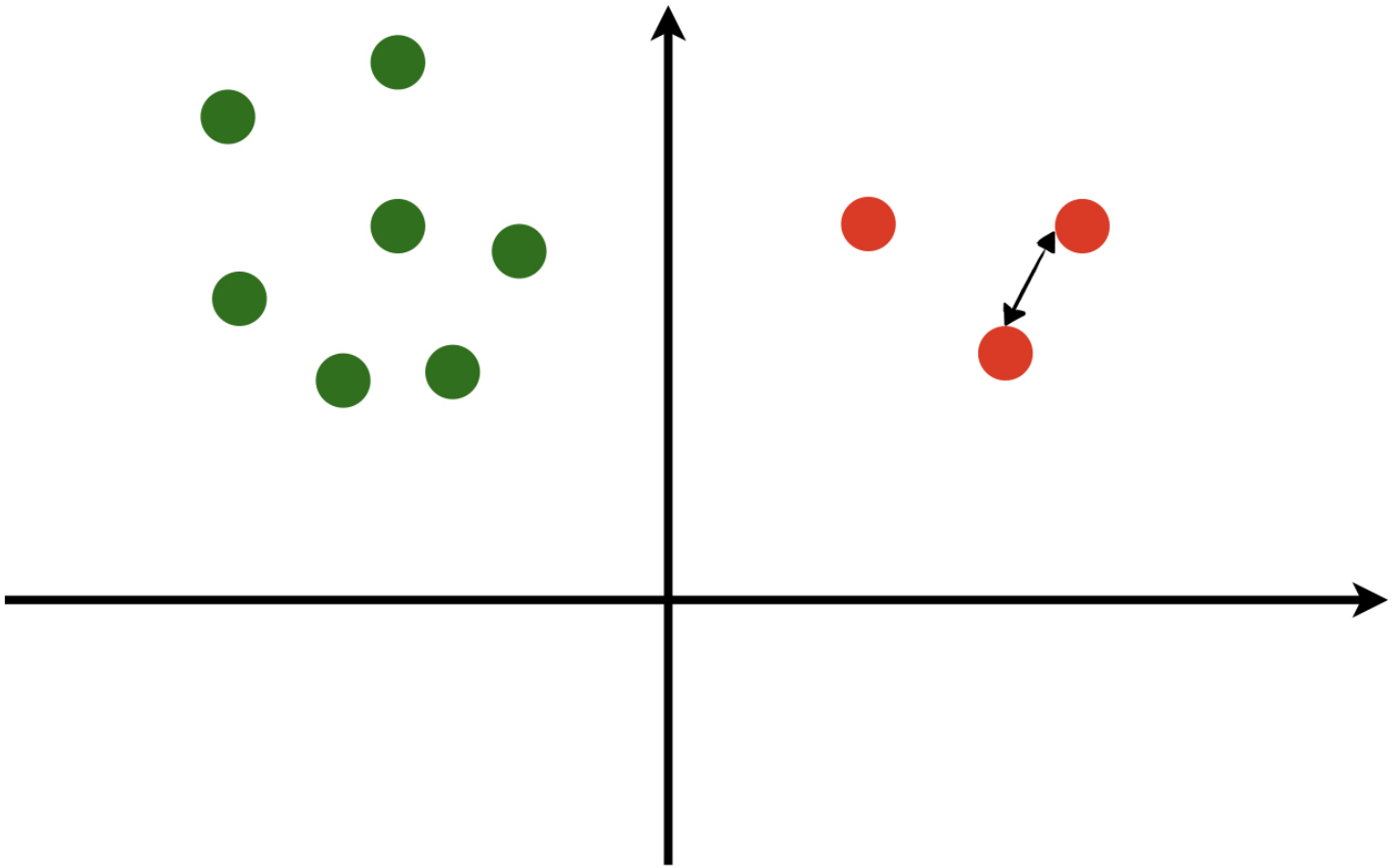
Quick Recap: *SVC Training*



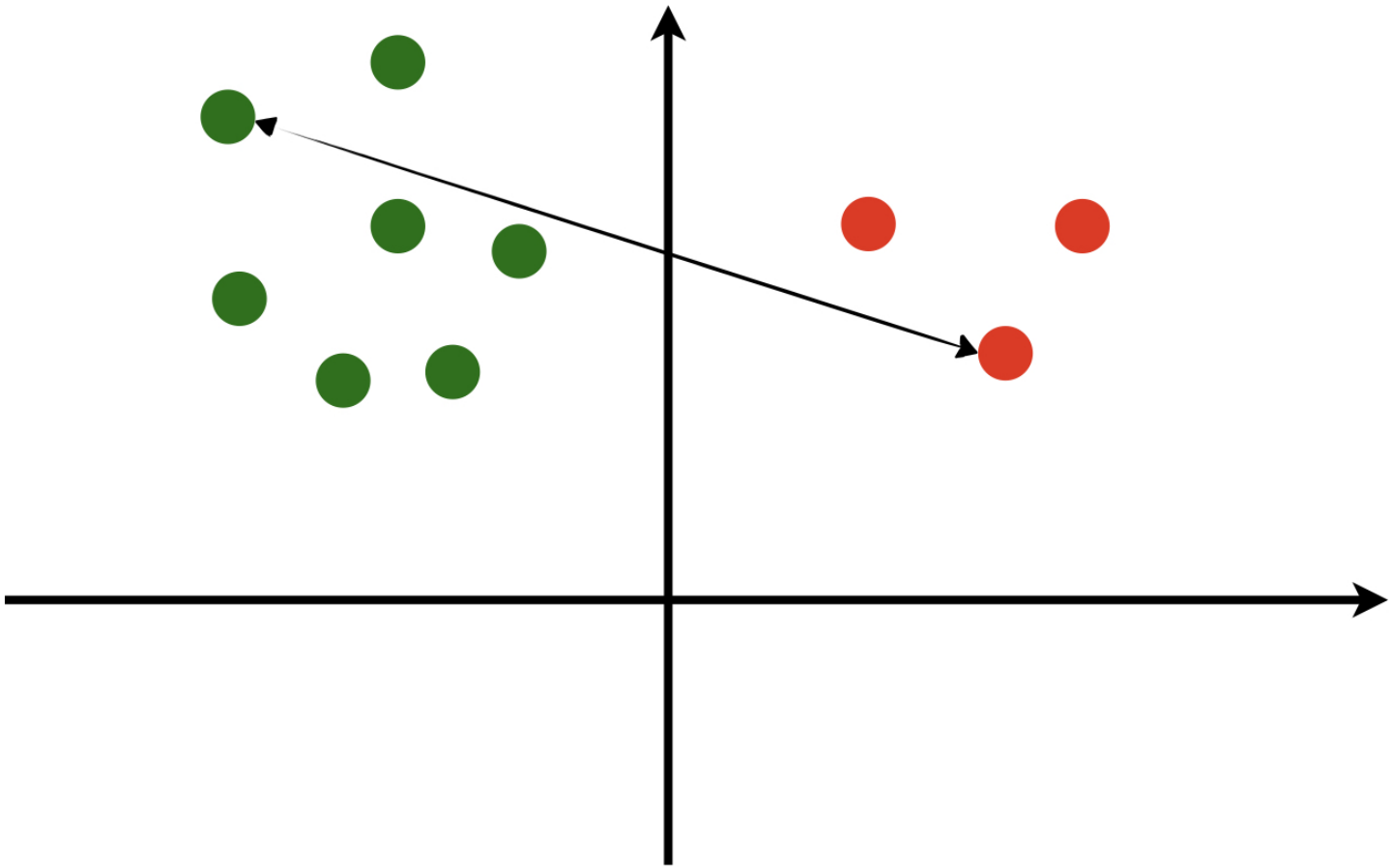
Quick Recap: *SVC Training*



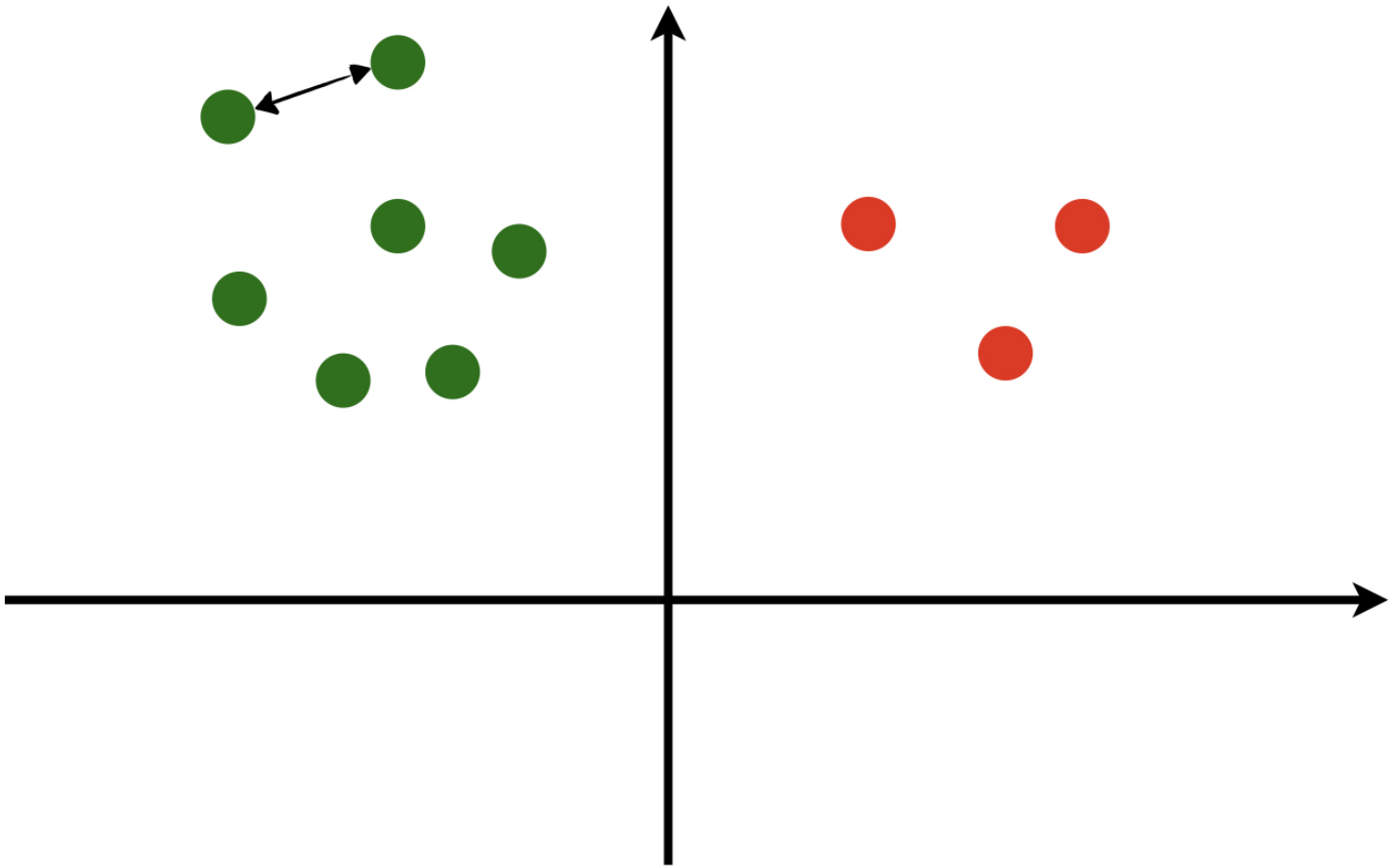
Quick Recap: *SVC Training*



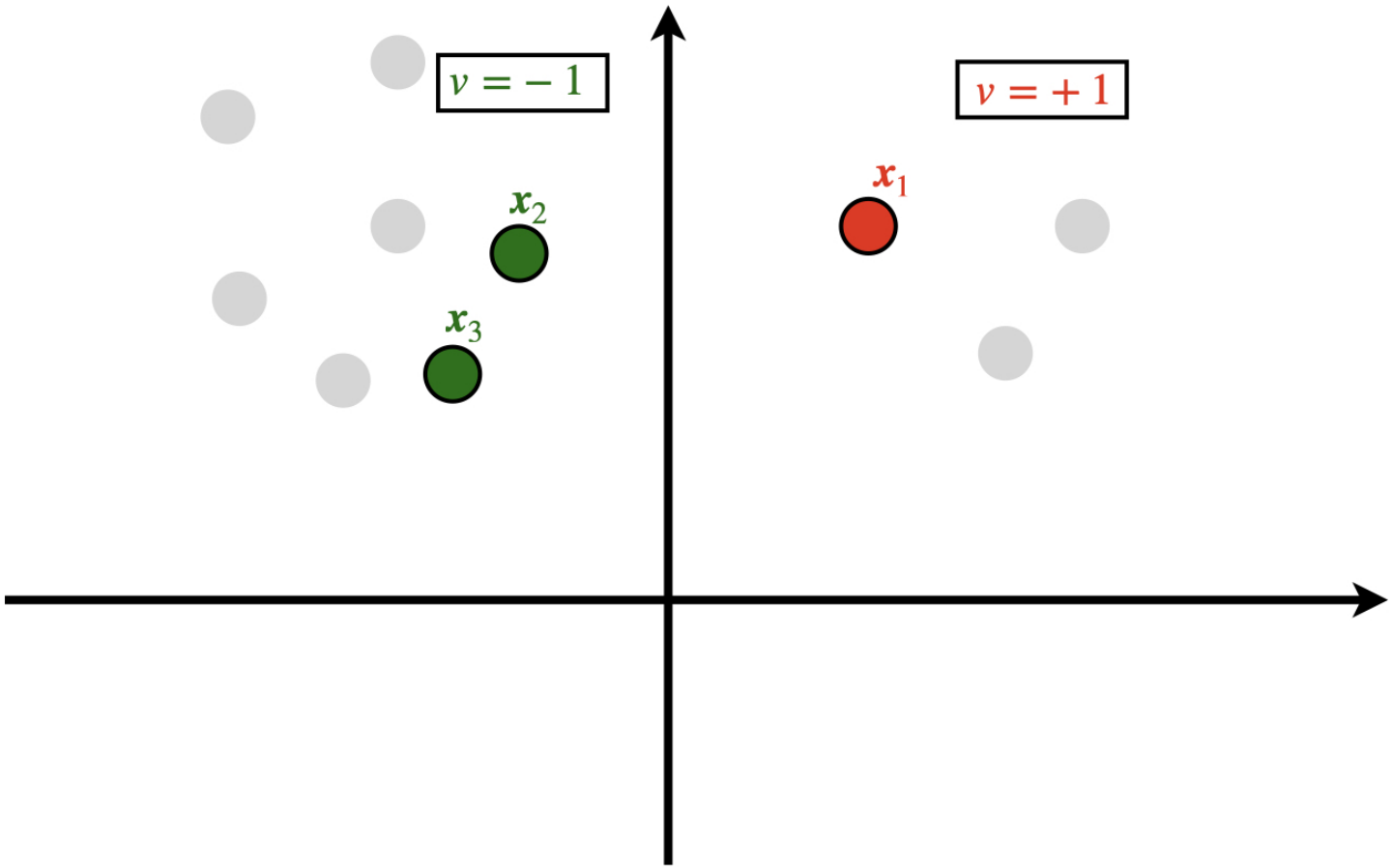
Quick Recap: *SVC Training*



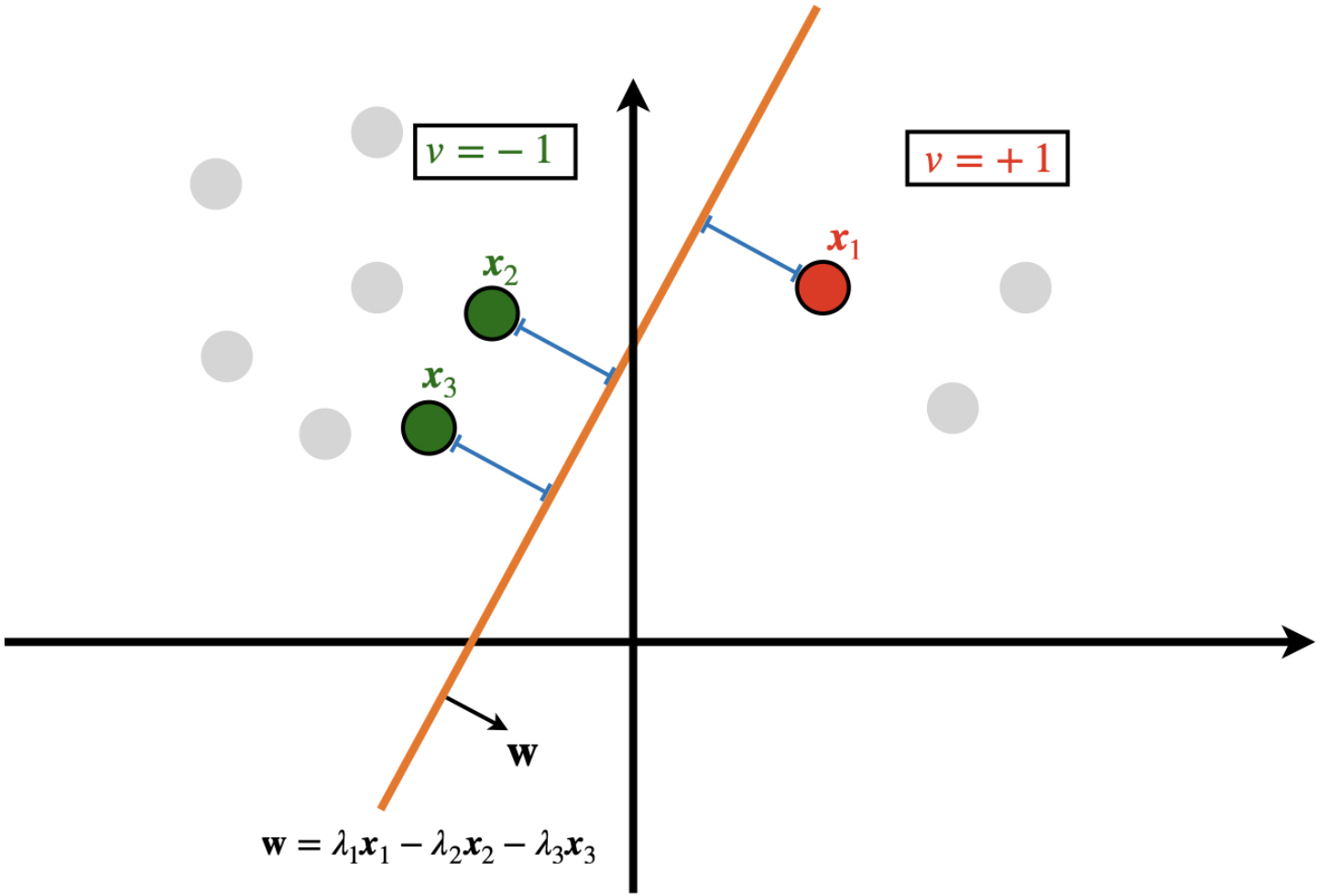
Quick Recap: *SVC Training*



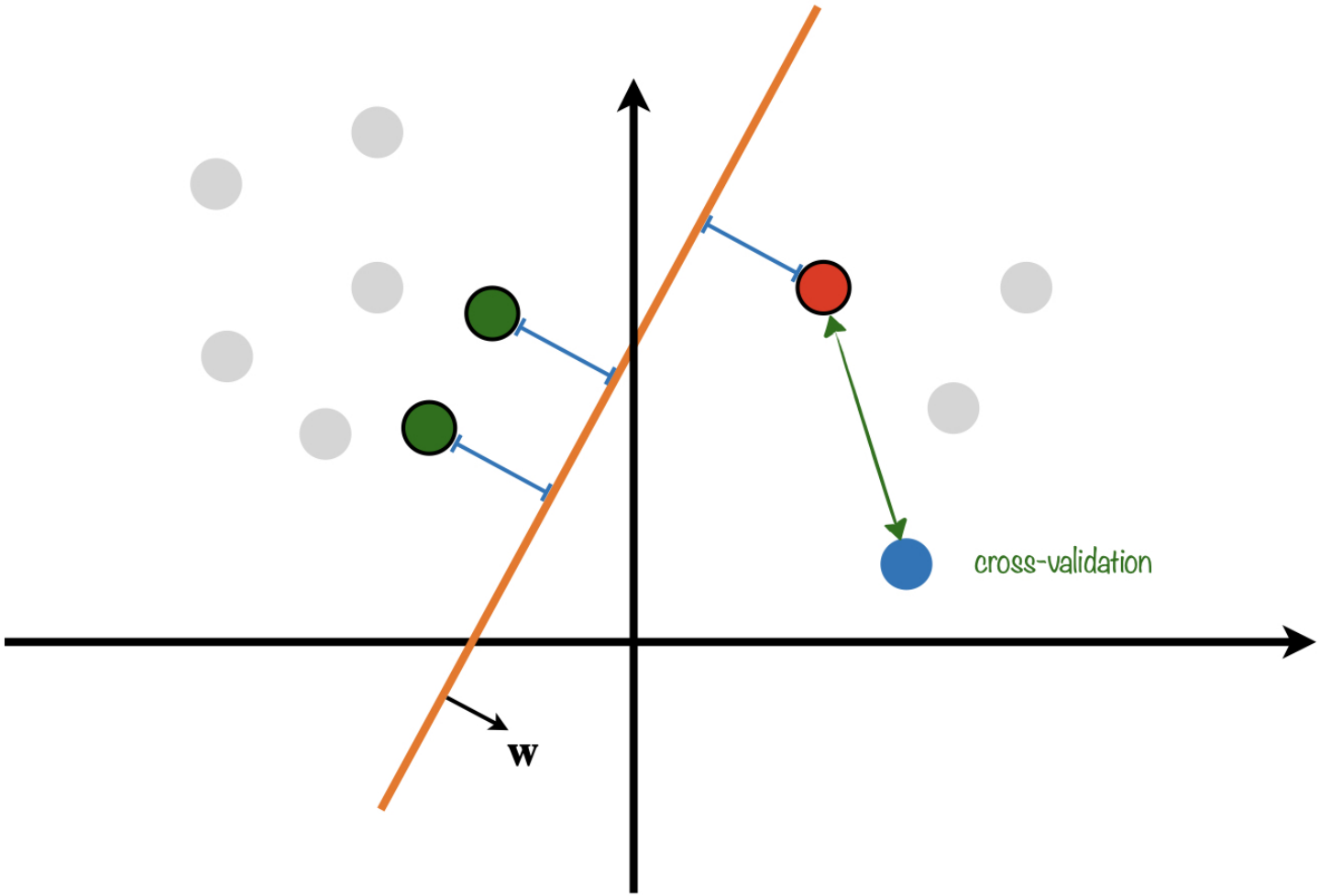
Quick Recap: *SVC Training*



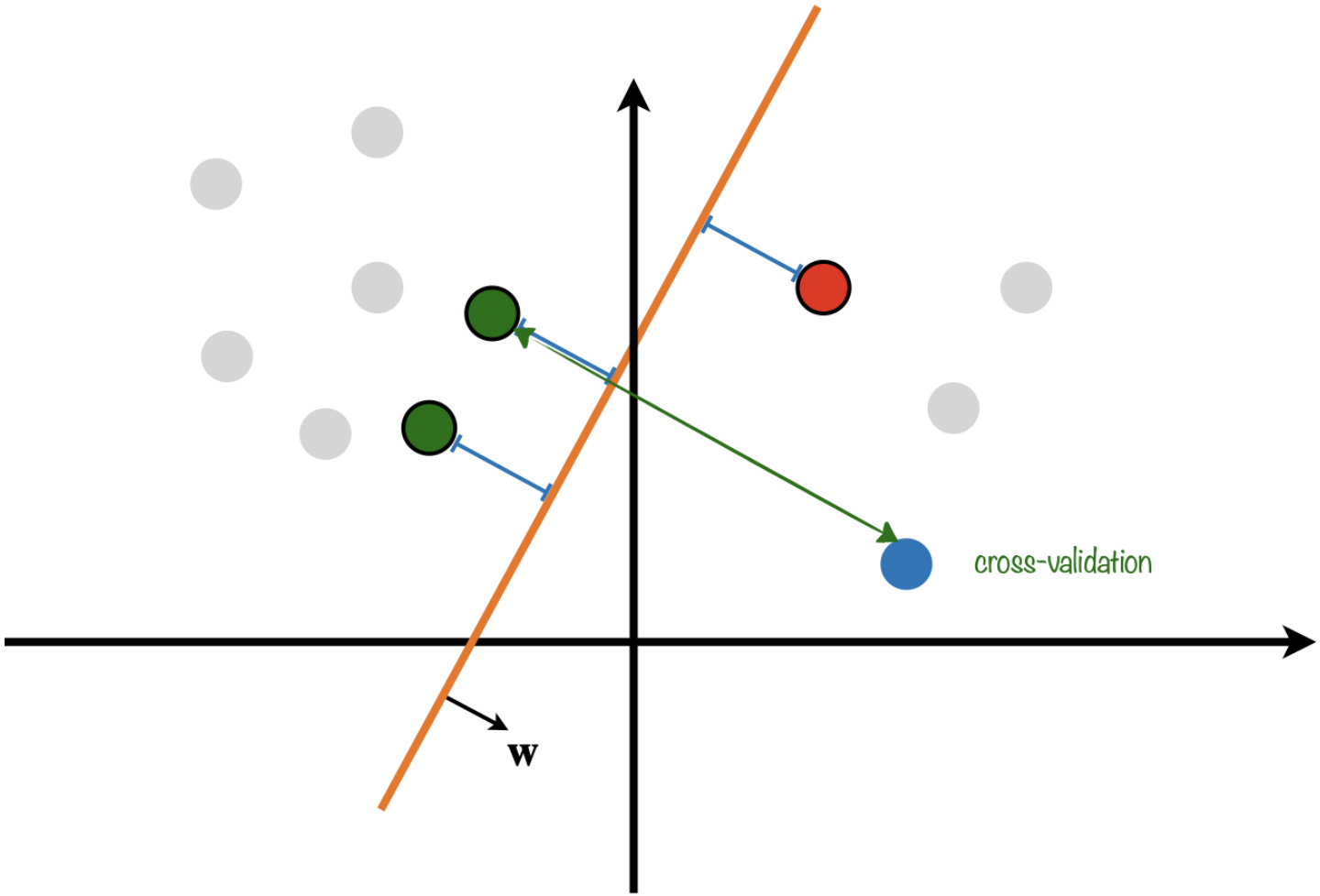
Quick Recap: *SVC Training*



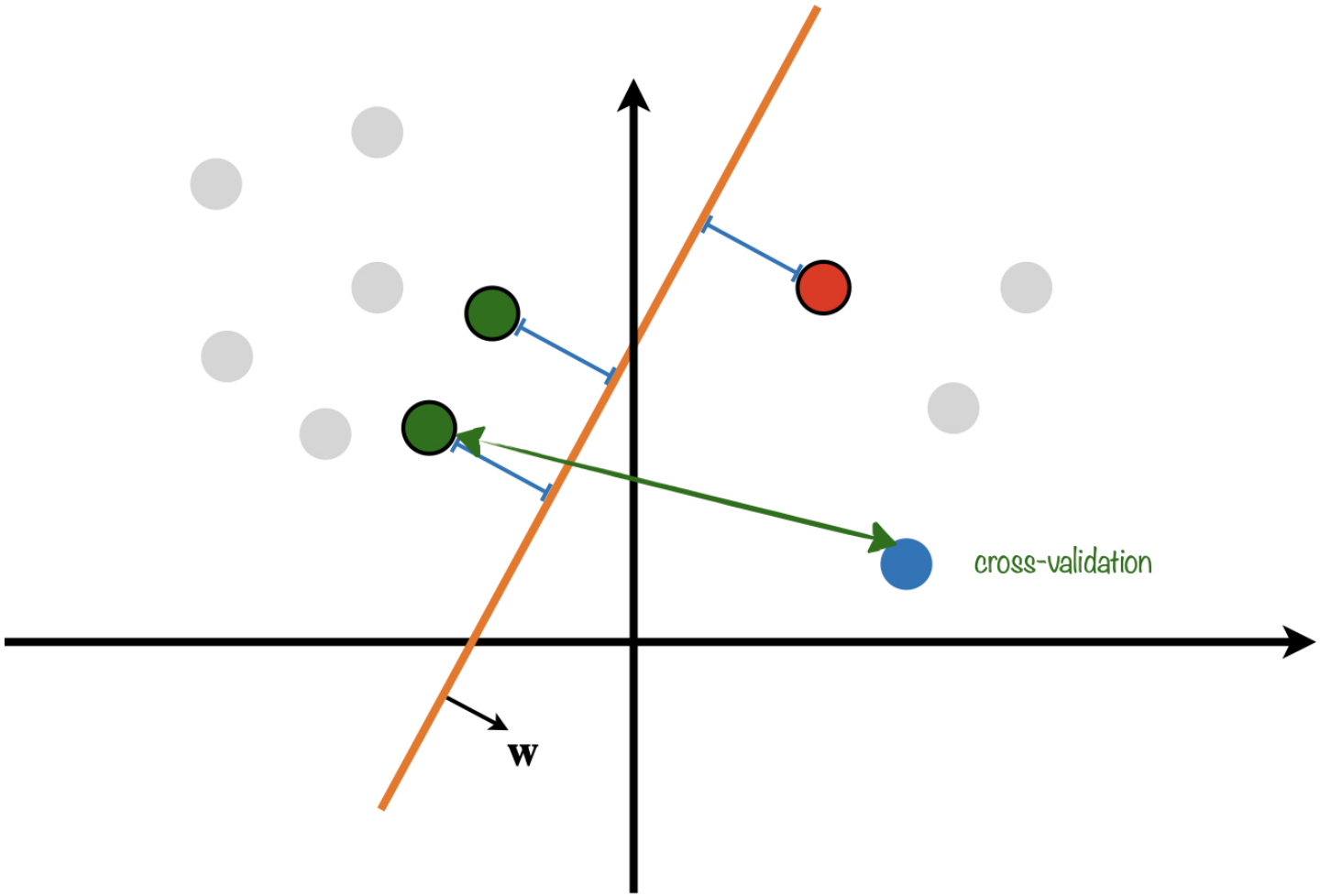
Quick Recap: *SVC Inference*



Quick Recap: *SVC Inference*



Quick Recap: *SVC Inference*



Today's Agenda: *Support Vector Machine*

Today, we extend the idea of SVC to nonlinear settings through

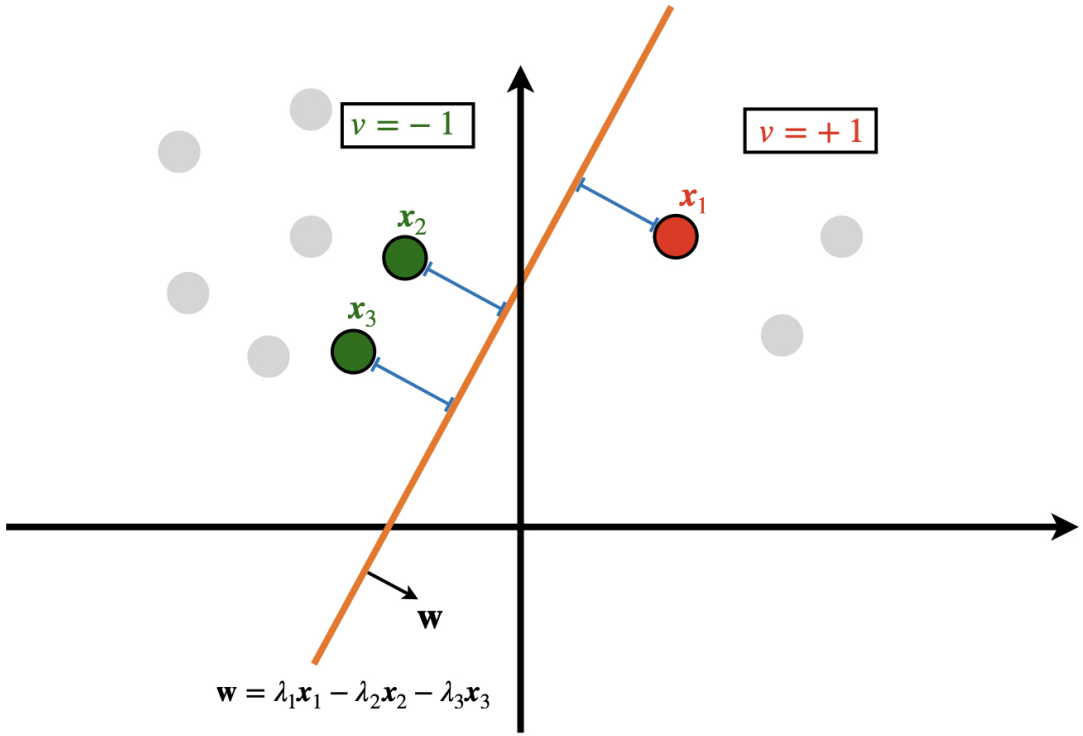
Support Vector Machine and Concept of Kernels

In this way, we discuss the following topics

- *We extend the idea of SVC to nonlinear patterns via embedding*
- *We develop SVM via the kernel trick*
- *We get the notions of kernel and representation learning*

SVC is Still Linear!

SVC can classify if it is linearly separable

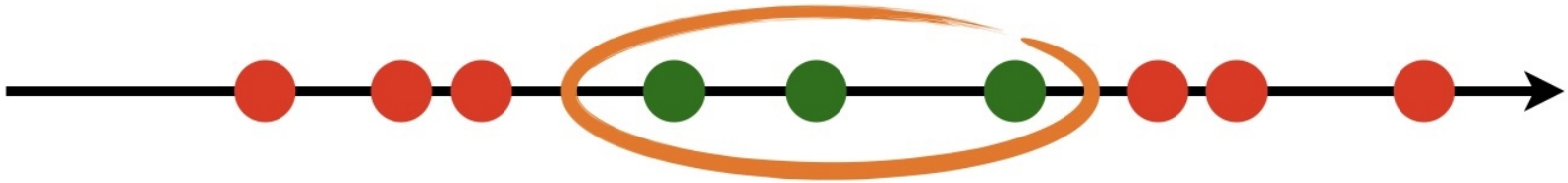


? *What's going to happen if data is not so simply structures?*

Example I: *Cat or Dog*

? *What if in our Cat or Dog example, the weights are sorted like this?*

! *Well, we need a nonlinear classifier then*



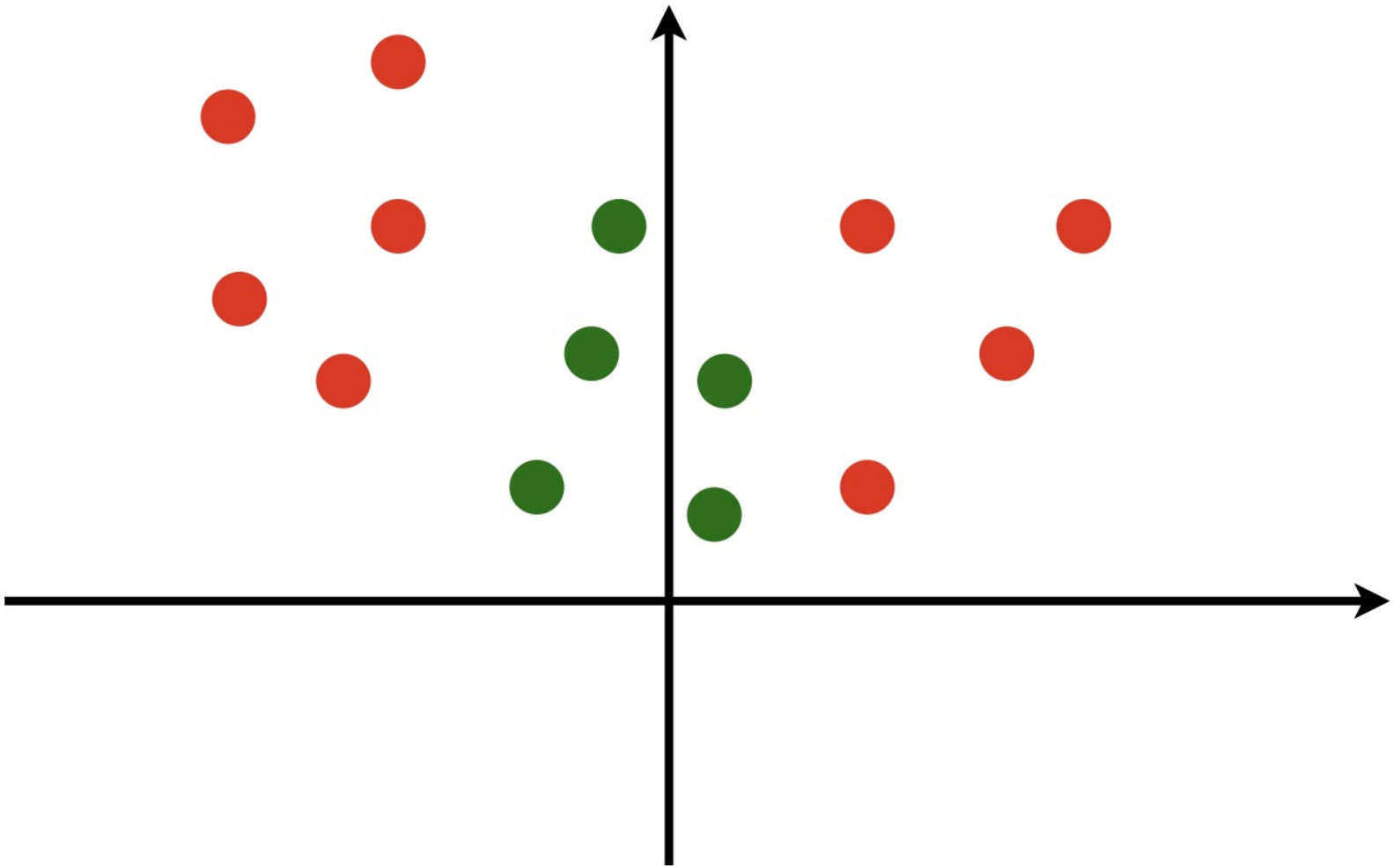
Nonlinear Classifier

Classifier that infers class of samples from nonlinear computations, e.g.,

$$z_n = w_1 x_n + w_2 x_n^2 \rightsquigarrow y_n = \operatorname{sign}(z_n)$$

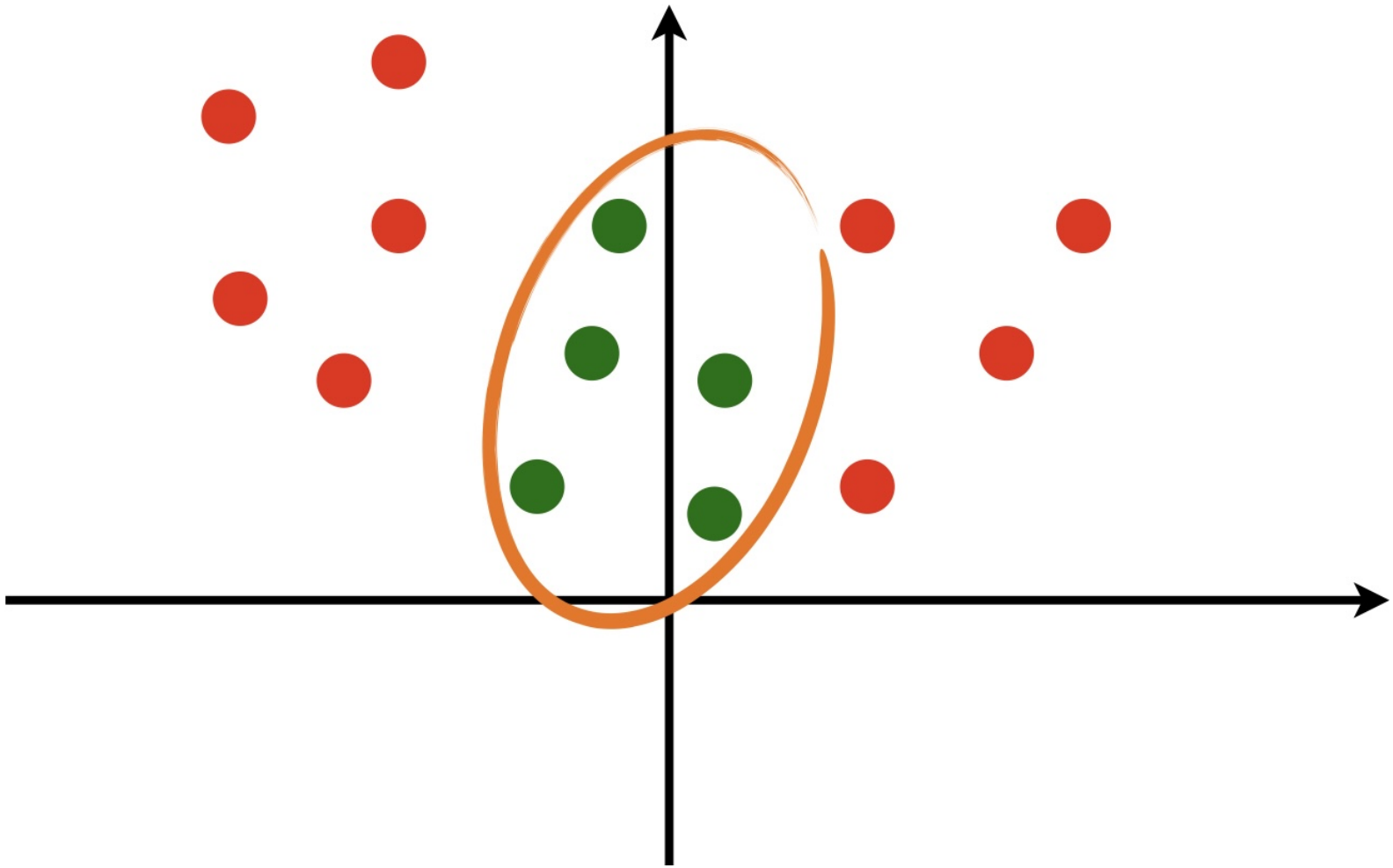
Example II: *Curved Boundary*

- ! *Such classifier should be needed in many problems*



Example II: *Curved Boundary*

- ! *Such classifier should be needed in many problems*



Example I: *Cat or Dog*

Let's get back to our Cat or Dog example: *We have a dataset*

$$\mathbb{D} = \{(x_n, v_n) : n = 1, \ldots, N\}$$

But now we cannot simply divide the green points from red ones by thresholding

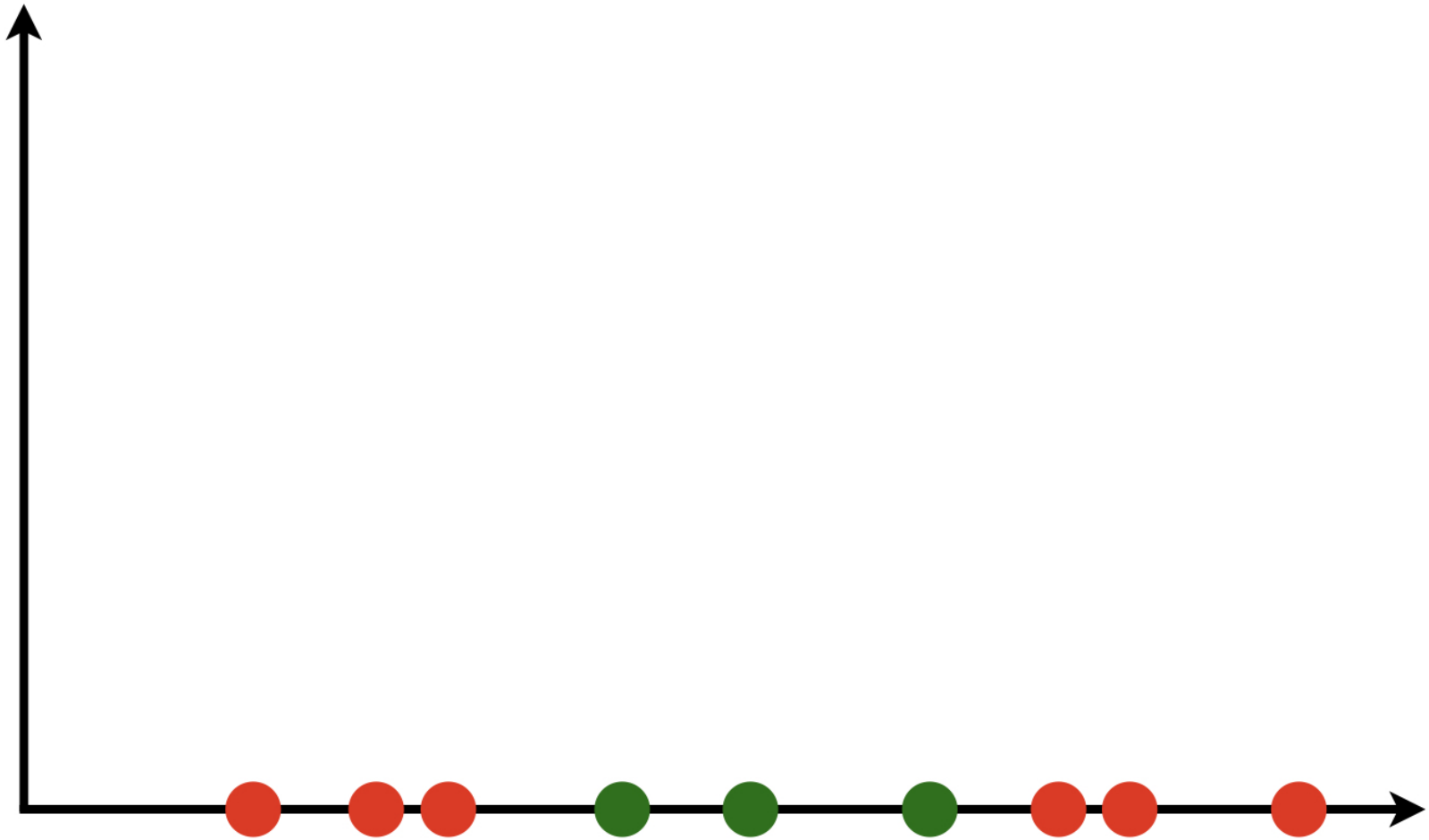
? *How can we do it in a systematic way then?*

Example I: *Cat or Dog*

Let's try a trick

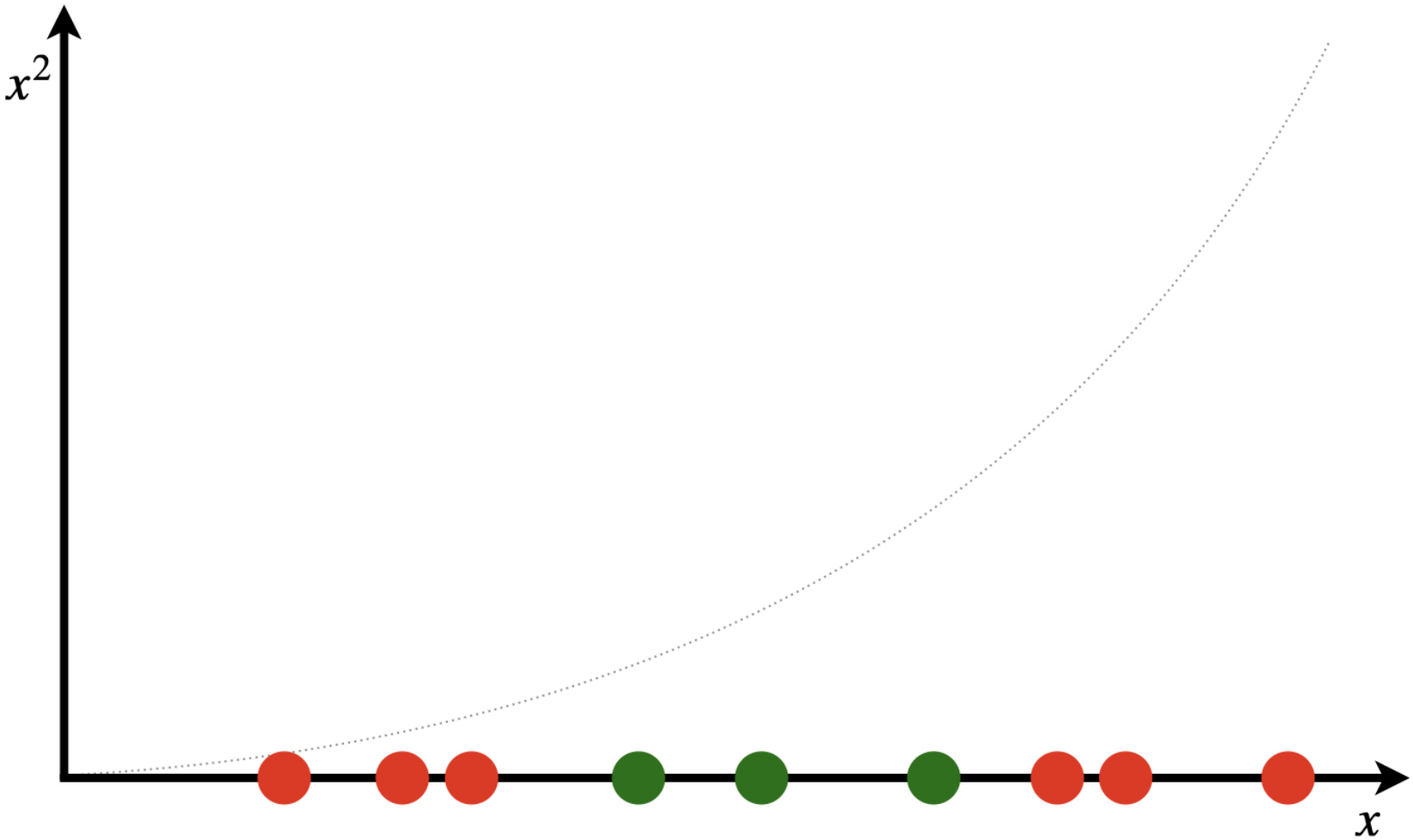
Example I: *Cat or Dog*

Let's try a trick: we add a second dimension to our data



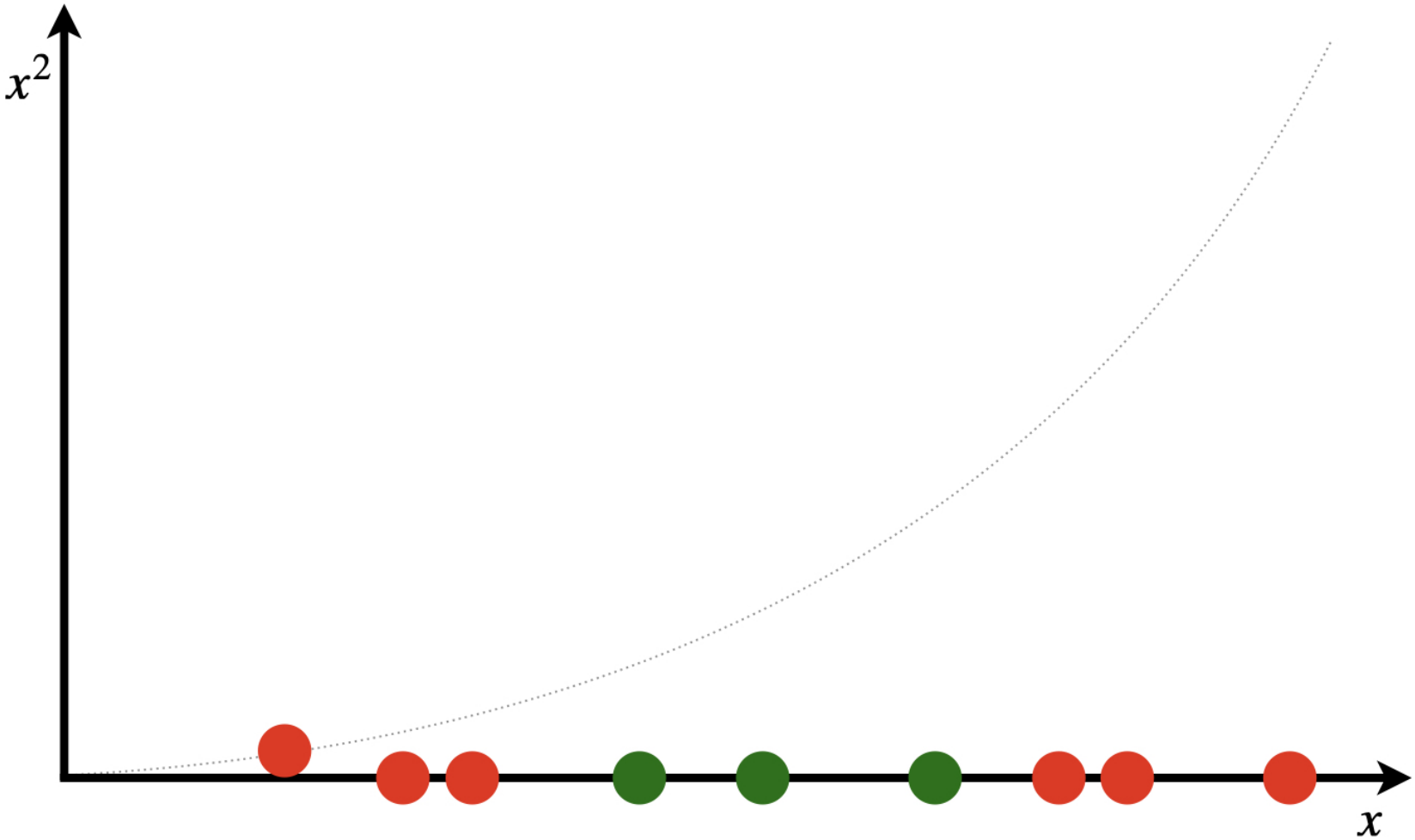
Example I: *Cat or Dog*

Let's try a trick: for this dimension we compute the square of samples



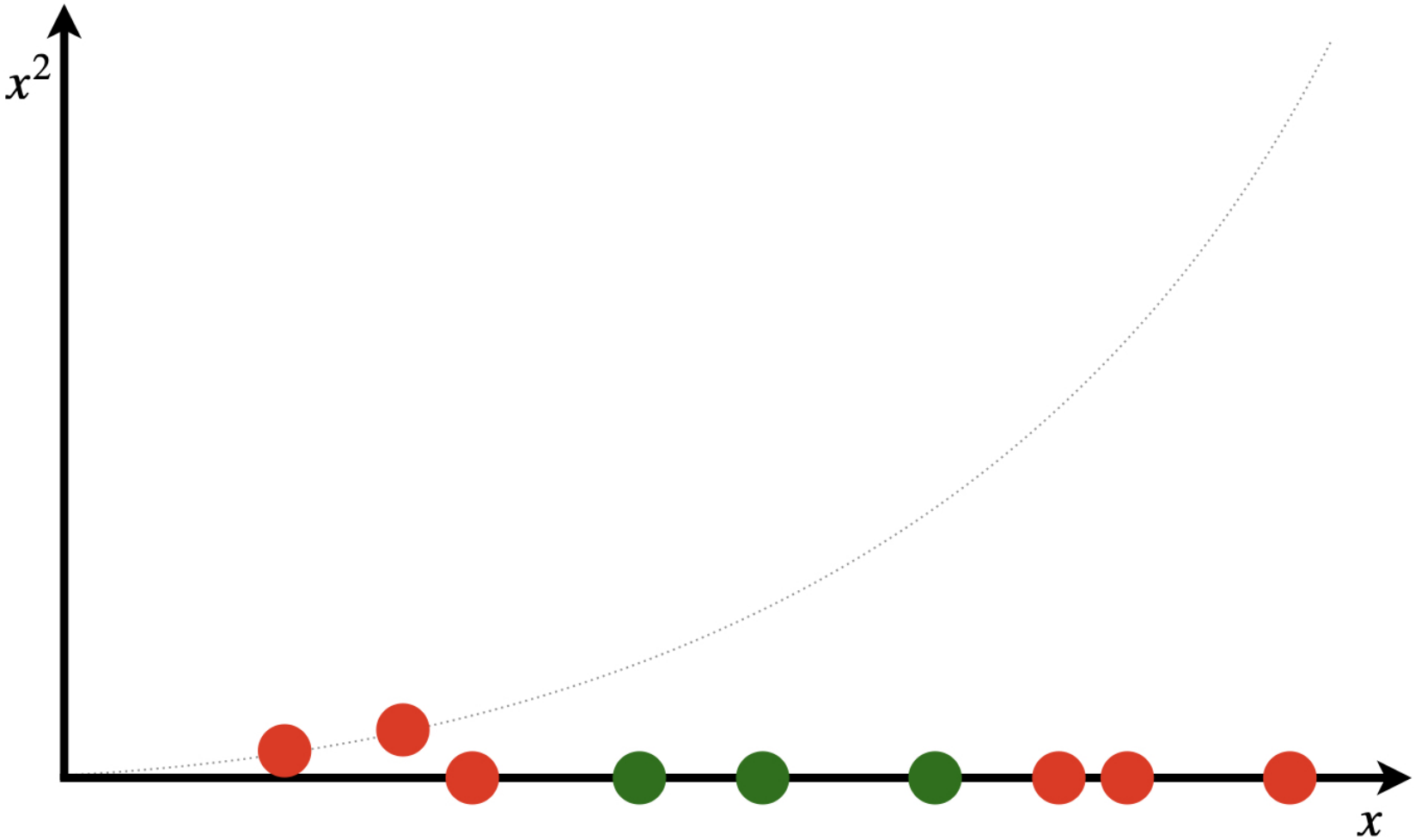
Example I: *Cat or Dog*

Let's try a trick: now we represent each point with a new vector $\tilde{\boldsymbol{x}} = [x, x^2]$



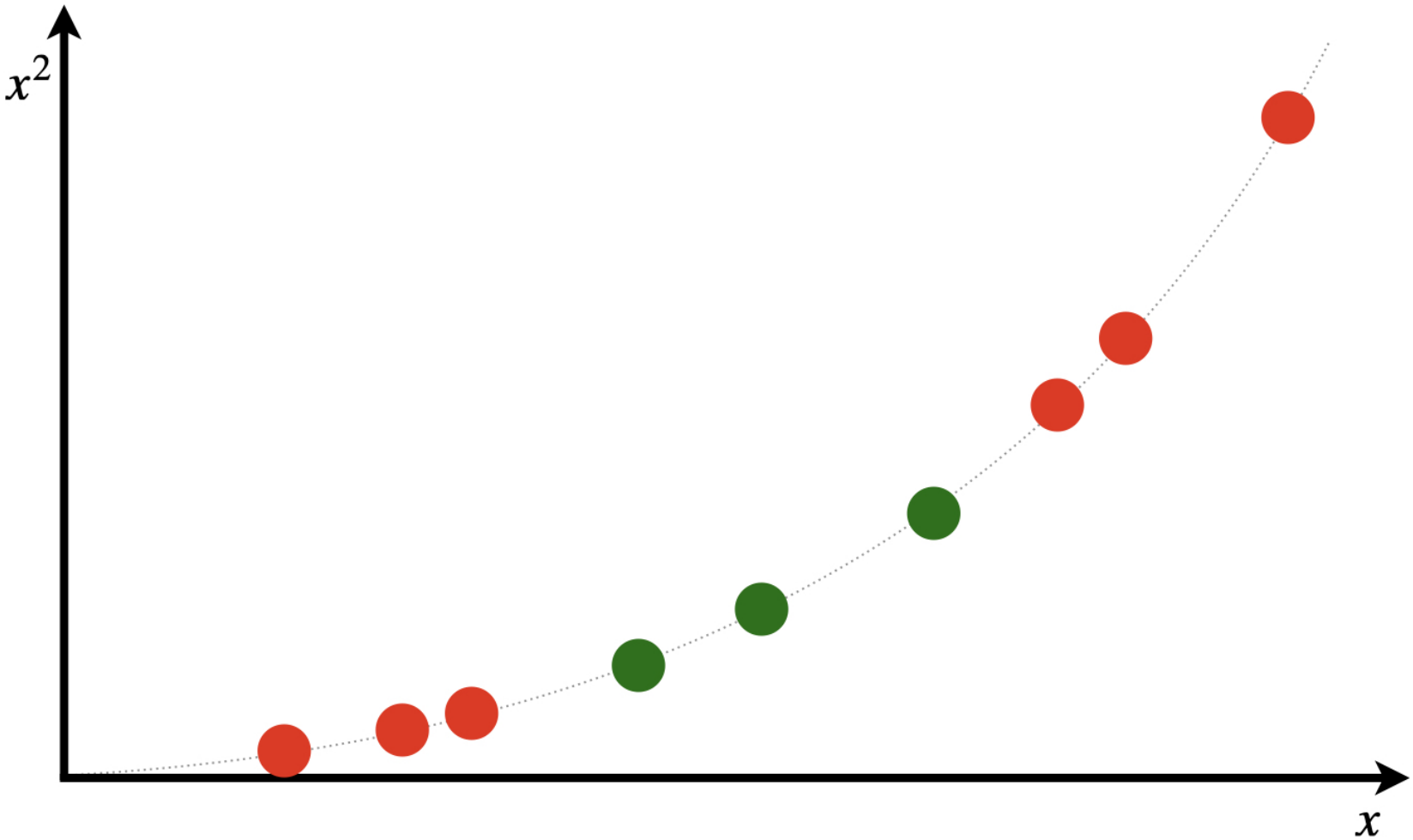
Example I: *Cat or Dog*

Let's try a trick: now we represent each point with a new vector $\tilde{\boldsymbol{x}} = [x, x^2]$



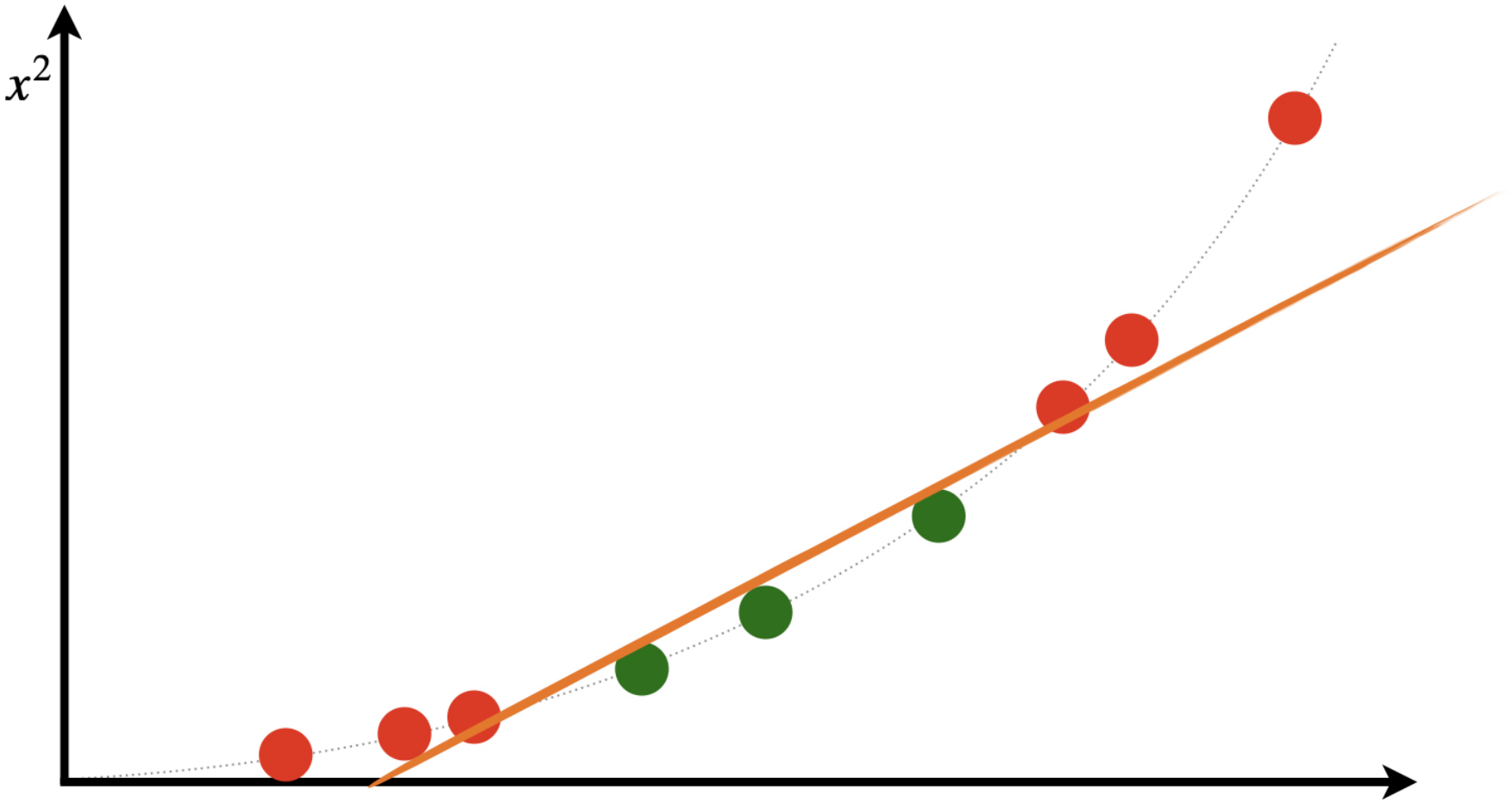
Example I: *Cat or Dog*

Let's try a trick: we transform the whole dataset



Example I: *Cat or Dog*

This transformed dataset can be linearly classified in 2D space!



Going Higher Dimensions

? *What is happening here?*

We see that the dataset cannot be classified perfectly by a linear model

We extract high-dimensional features as

$$x \mapsto \tilde{\boldsymbol{x}} = \varphi(x) = \begin{bmatrix} x \\ x^2 \end{bmatrix}$$

and see that

$$\mathbb{D} = \{(\tilde{\boldsymbol{x}}_n, v_n) : n = 1, \ldots, N\}$$

is classified by linear model!

We could make a nonlinear problem linear in higher dimensions

Nonlinear Low-Dimension $\leftrightsquigarrow$ Linear High-Dimension

This suggests a generic recipe: *for a nonlinear problem with data*

$$\mathbb{D} = \{(\boldsymbol{x}_n, v_n) : n = 1, \ldots, N\}$$

first extract high-dimensional features

Feature Extraction: *Embedding*

Use an embedding function φ to map samples to high-dimensional features

$$\boldsymbol{x}_n \in \mathbb{R}^d \mapsto \varphi(\boldsymbol{x}_n) \in \mathbb{R}^D$$

where $D > d$

Nonlinear Low-Dimension ⟷ Linear High-Dimension

This suggests a generic recipe: *for a nonlinear problem with data*

$$\mathbb{D} = \{(\boldsymbol{x}_n, v_n) : n = 1, \ldots, N\}$$

then find a linear classifier for high-dimensional features

Feature Classification

Find a SVC with weight $\mathbf{w} \in \mathbb{R}^D$ *for* $\varphi(\boldsymbol{x}_n)$

$$y_n = \operatorname{sign}\left(\mathbf{w}^\mathsf{T} \varphi(\boldsymbol{x}_n)\right)$$

Nonlinear Low-Dimension $\leftrightsquigarrow$ Linear High-Dimension

? *How to use the model for inferring class of a new data?*

We infer based on the high-dimensional features

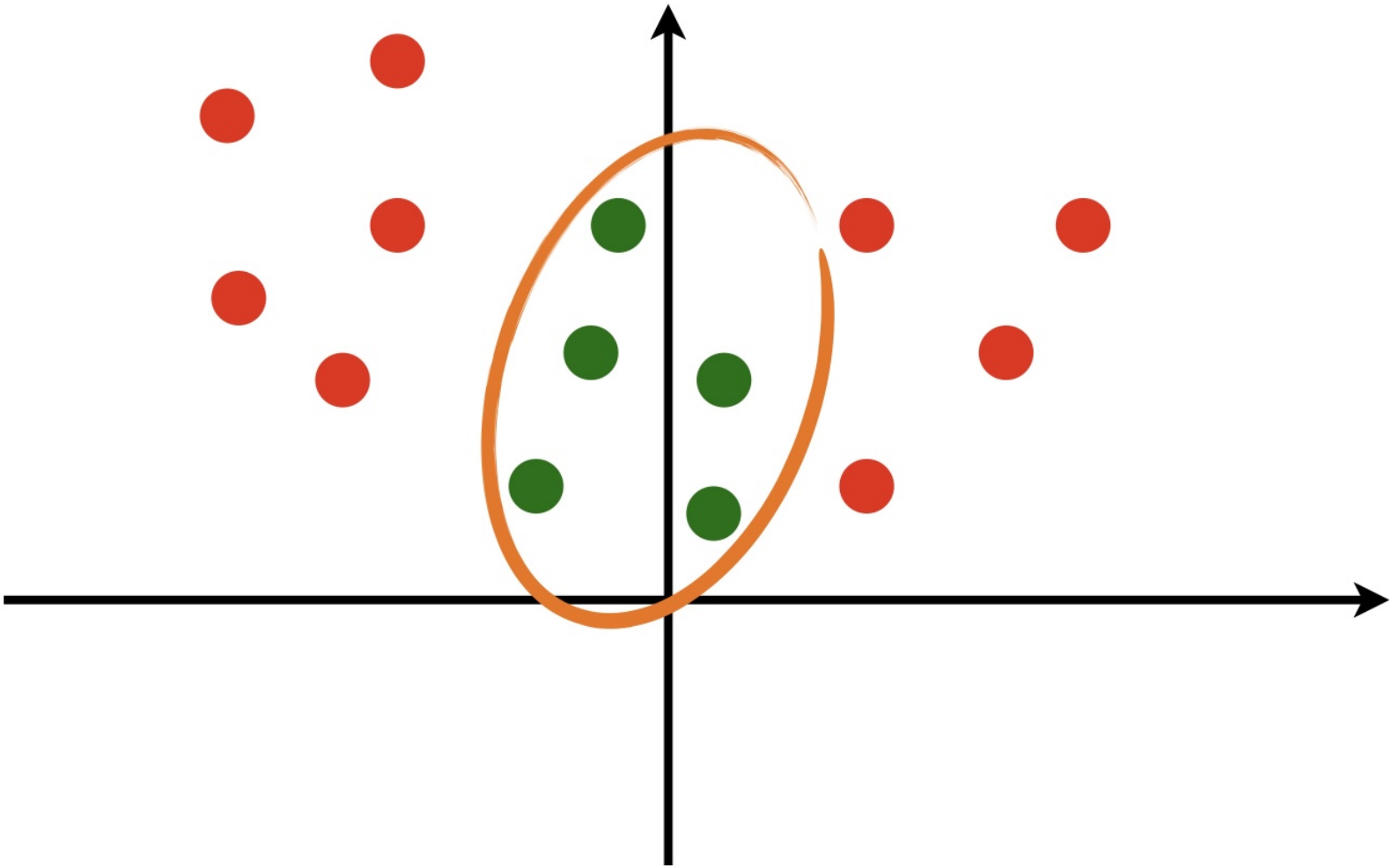
Inference

The label of the new sample $\boldsymbol{x}$ *is given as*

$$y = \text{sign}\left(\mathbf{w}^{\mathsf{T}} \varphi\left(\boldsymbol{x}\right)\right)$$

Example II: *Curved Boundary*

Let's look at the round boundary visual example

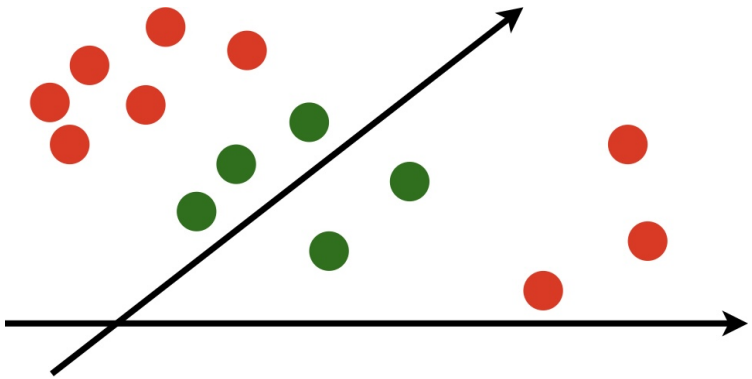


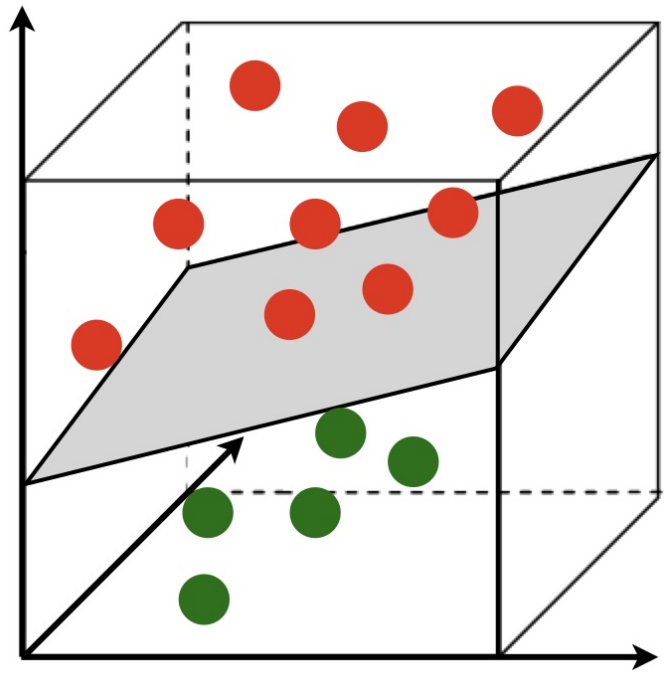
Example II: *Curved Boundary*

The points can be linearly separable in 3D

maps them to 3D as $\varphi(\boldsymbol{x}_n)$

get the points $\boldsymbol{x}_n$ *in 2D*





Support Vector Machine: *Nonlinear SVC*

Support Vector Machine

Support vector machine (SVM) is an SVC that learns from high-dimensional features extract by an embedding function $\varphi : \mathbb{R}^d \mapsto \mathbb{R}^D$

↳ *It can learn nonlinear patterns*

Though sounds promising, it seems a bit challenging

- ? *What should be the embedding function?*
- ? *What if the model need super high-dimensional features?*
  - ↳ *This means that we should work in very high dimensions*
  - ↳ *We even don't know how large it should be!*

Recall: *SVC Training and Inference*

To train an SVC, we solve the dual problem

$$\max_{\boldsymbol{\lambda} \geqslant \mathbf{0}} \sum_{n=1}^{N} \lambda_n - \frac{1}{2} \sum_{n=1}^{N} \sum_{m=1}^{N} \lambda_n \lambda_m \upsilon_n \upsilon_m \boldsymbol{x}_n^\mathsf{T} \boldsymbol{x}_m$$

Once we found the dual values $\lambda_n^\star$, we classify as

$$\mathbf{w}^\star = \sum_{n=1}^{N} \lambda_n^\star \upsilon_n \boldsymbol{x}_n \rightsquigarrow y = \operatorname{sign}\left(\sum_{n=1}^{N} \lambda_n^\star \upsilon_n \boldsymbol{x}_n^\mathsf{T} \boldsymbol{x} \right)$$

Recall: Only Cross-Validations

Recall that we only need the correlations $\boldsymbol{x}_m^\mathsf{T} \boldsymbol{x}_n$

SVC with Embedded Features

For SVM, we would do the same: *we find the dual values through*

$$\max_{\boldsymbol{\lambda} \geqslant \mathbf{0}} \sum_{n=1}^{N} \lambda_n - \frac{1}{2} \sum_{n=1}^{N} \sum_{m=1}^{N} \lambda_n \lambda_m v_n v_m \varphi\left(\boldsymbol{x}_n\right)^{\mathsf{T}} \varphi\left(\boldsymbol{x}_m\right)$$

We then infer the class of a new sample as

$$\mathbf{w}^{\star} = \sum_{n=1}^{N} \lambda_n^{\star} v_n \varphi\left(\boldsymbol{x}_n\right) \rightsquigarrow y = \operatorname{sign}\left(\sum_{n=1}^{N} \lambda_n^{\star} v_n \varphi\left(\boldsymbol{x}_n\right)^{\mathsf{T}} \varphi\left(\boldsymbol{x}\right) \right)$$

Again: Only Cross-Validations

Here again we only need the correlations $\varphi\left(\boldsymbol{x}_m\right)^{\mathsf{T}} \varphi\left(\boldsymbol{x}_n\right)$

Kernel Trick

We don't really need to work in high-dimensional space:

It's enough to know how the embeddings are correlated!

Kernel

A function $\mathcal{K} : \mathbb{R}^d \times \mathbb{R}^d \mapsto \mathbb{R}$ that computes the cross-correlation between high-dimensional features of two samples

$$\mathcal{K}(\boldsymbol{x}_m, \boldsymbol{x}_n) = \varphi(\boldsymbol{x}_n)^\mathsf{T} \varphi(\boldsymbol{x}_m)$$

We don't really need the embedding function φ:

we only need the kernel!

Kernel Trick: *Extended Cross-Validation*

This is called the kernel trick: *choose a kernel $\mathcal{K}$ and solve*

$$\max_{\boldsymbol{\lambda} \geqslant \mathbf{0}} \sum_{n=1}^{N} \lambda_n - \frac{1}{2} \sum_{n=1}^{N} \sum_{m=1}^{N} \lambda_n \lambda_m \upsilon_n \upsilon_m \mathcal{K}\left(\boldsymbol{x}_m, \boldsymbol{x}_n\right)$$

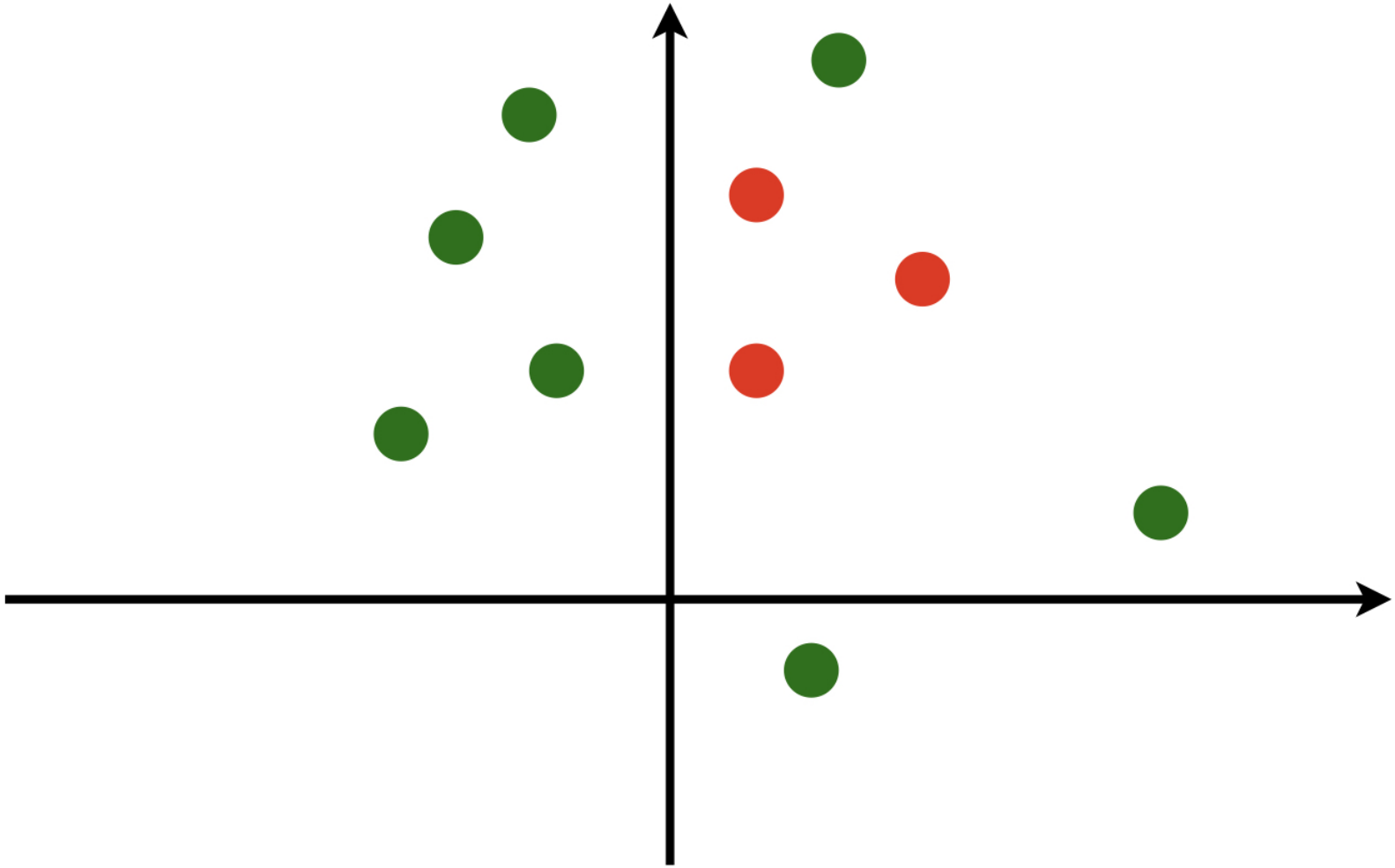
Once $\lambda_n^\star$ found classify by the same kernel $\mathcal{K}$ as

$$\mathbf{w}^\star = \sum_{n=1}^{N} \lambda_n^\star \upsilon_n \varphi\left(\boldsymbol{x}_n\right) \rightsquigarrow y = \operatorname{sign}\left(\sum_{n=1}^{N} \lambda_n^\star \upsilon_n \mathcal{K}\left(\boldsymbol{x}_n, \boldsymbol{x}\right)\right)$$

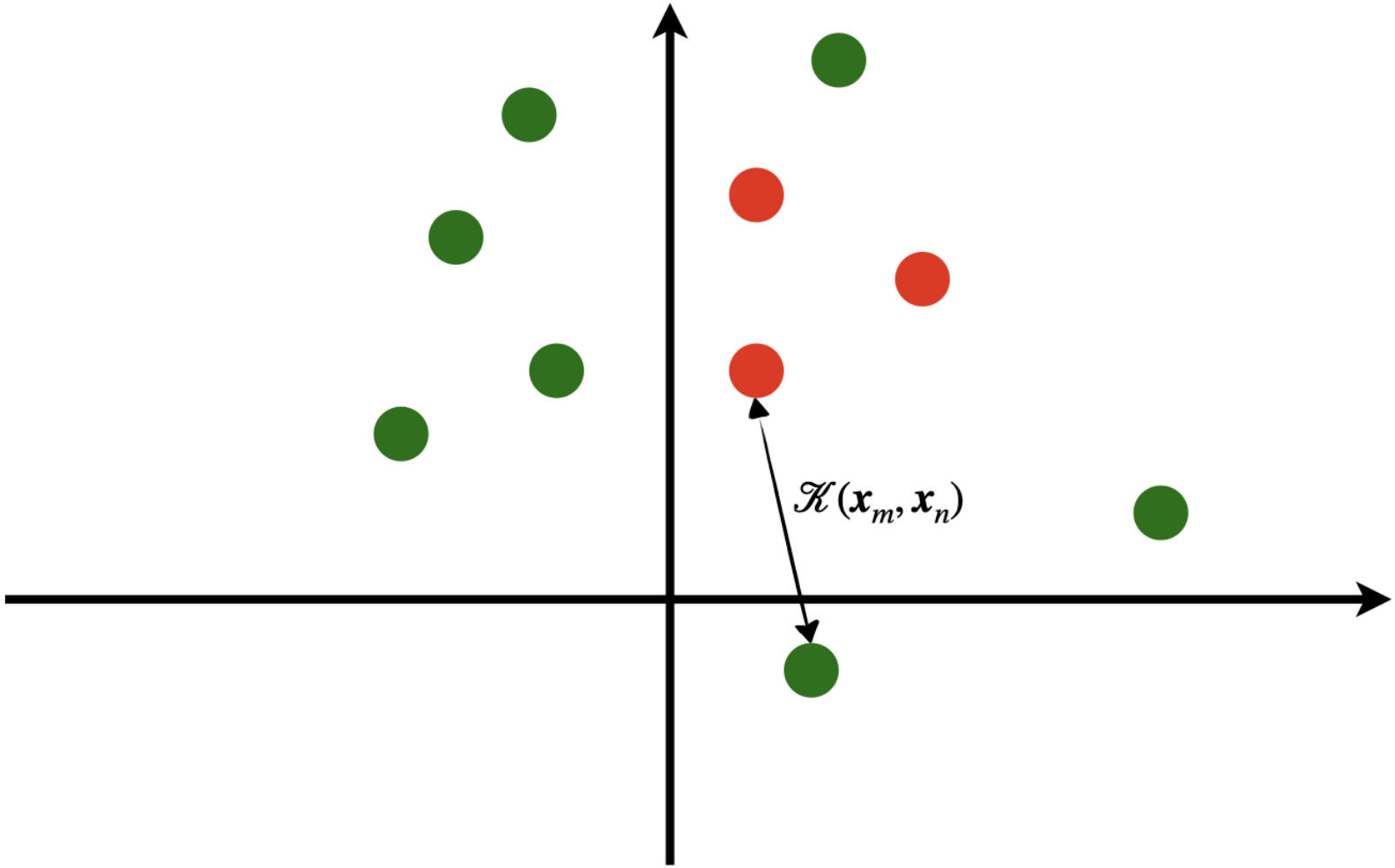
Moral of Story

SVM does what SVC do using a nonlinear (potentially very complicated) kernel

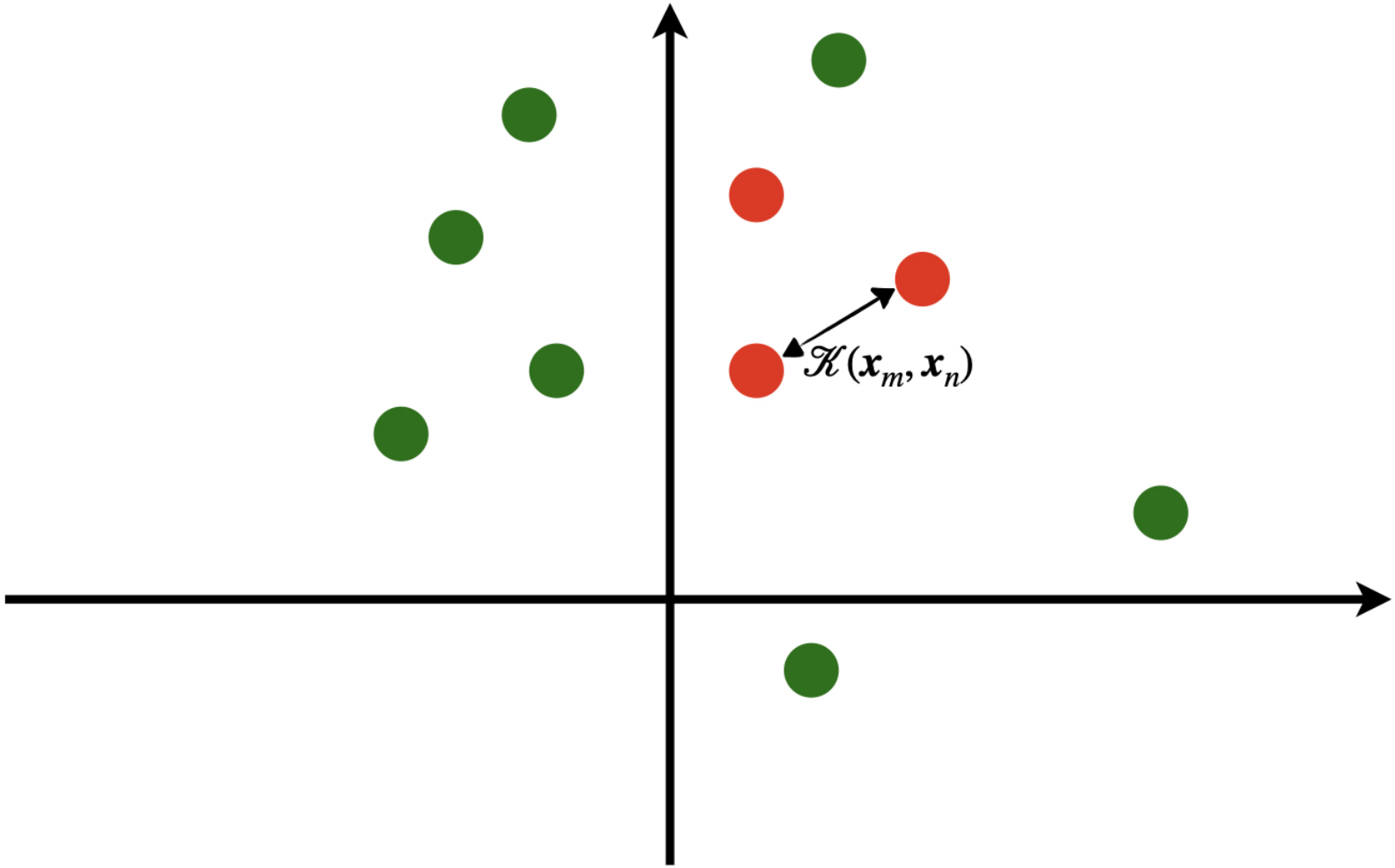
Support Vector Machines: *Visualization*



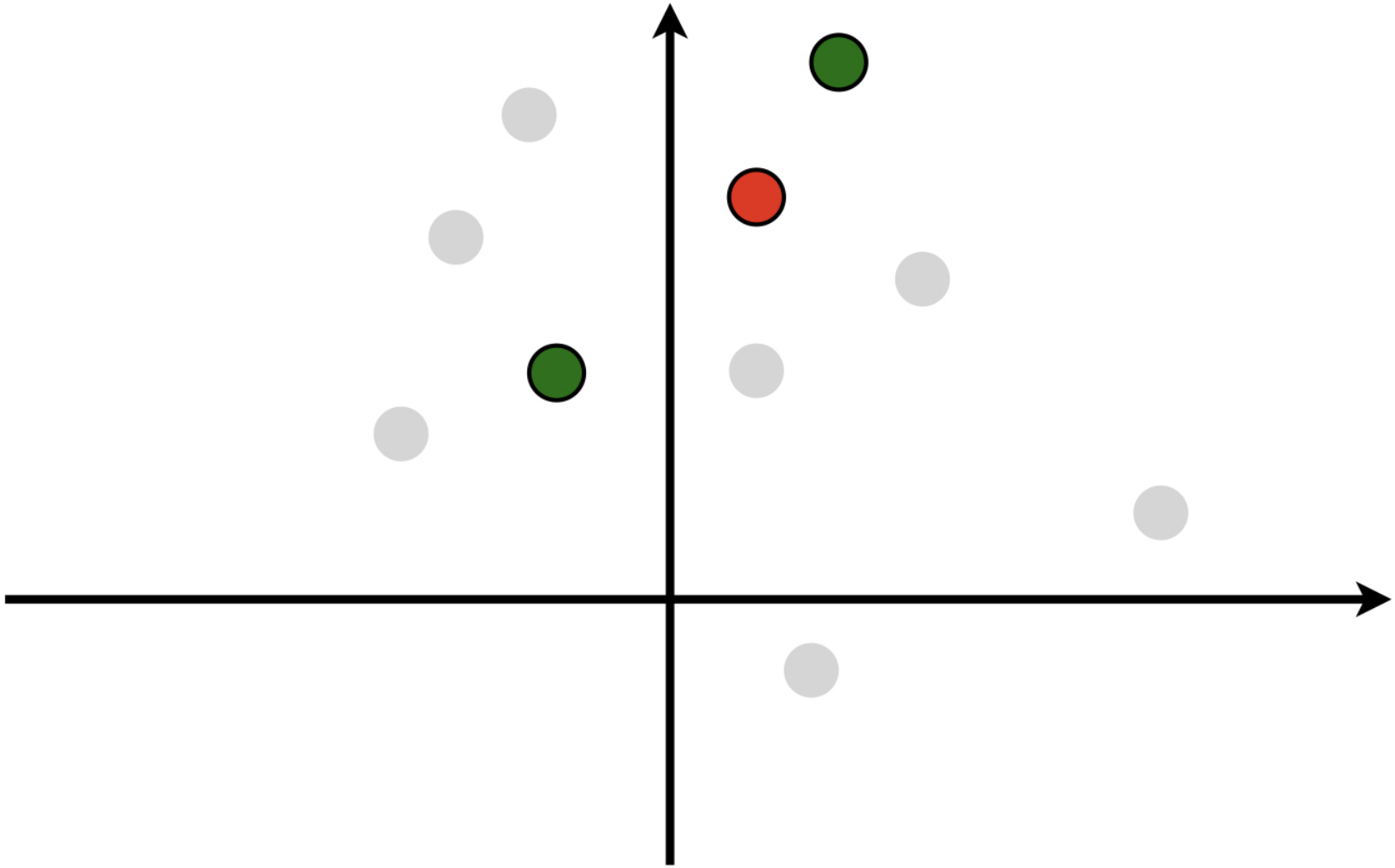
Support Vector Machines: *Visualization*



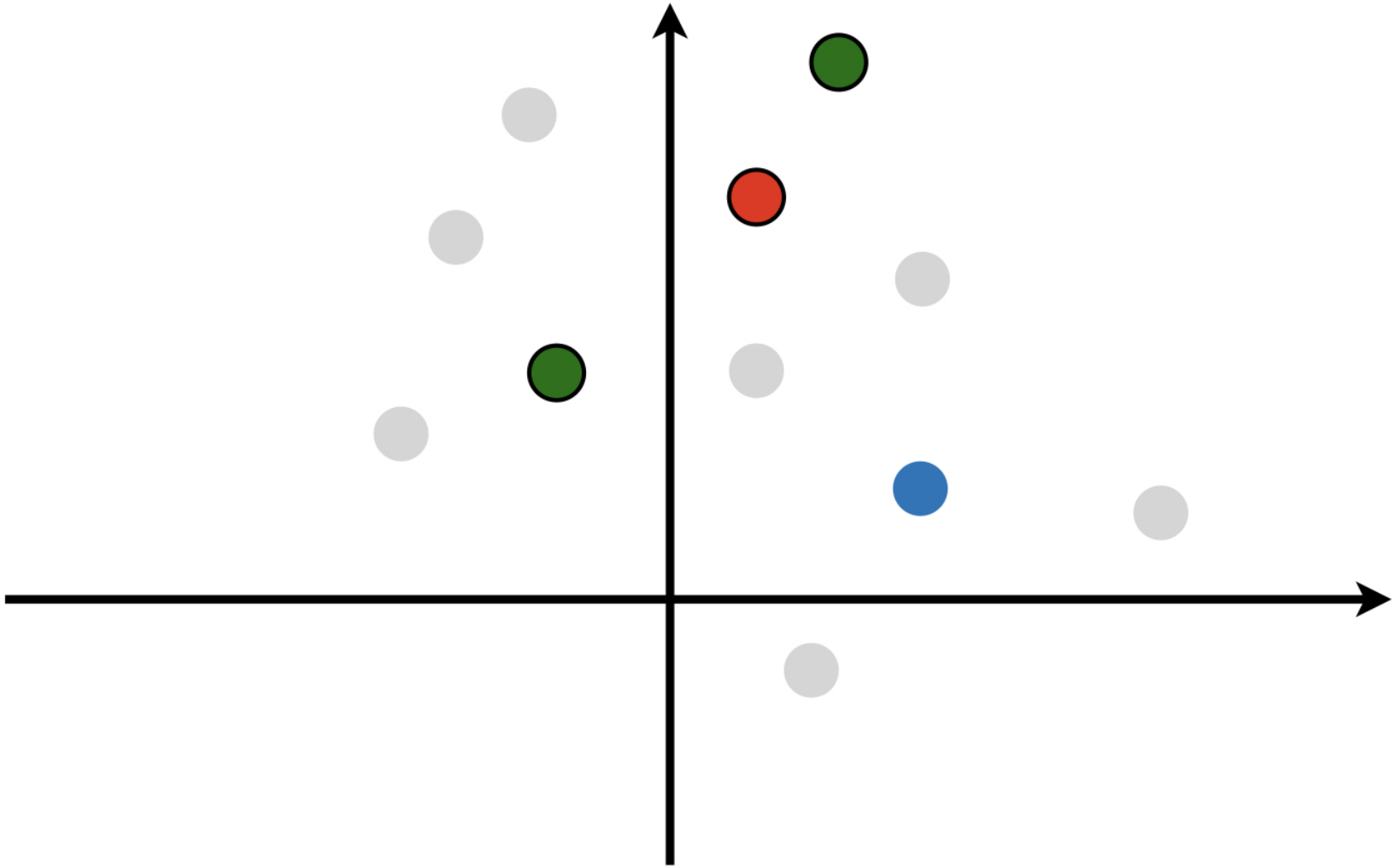
Support Vector Machines: *Visualization*



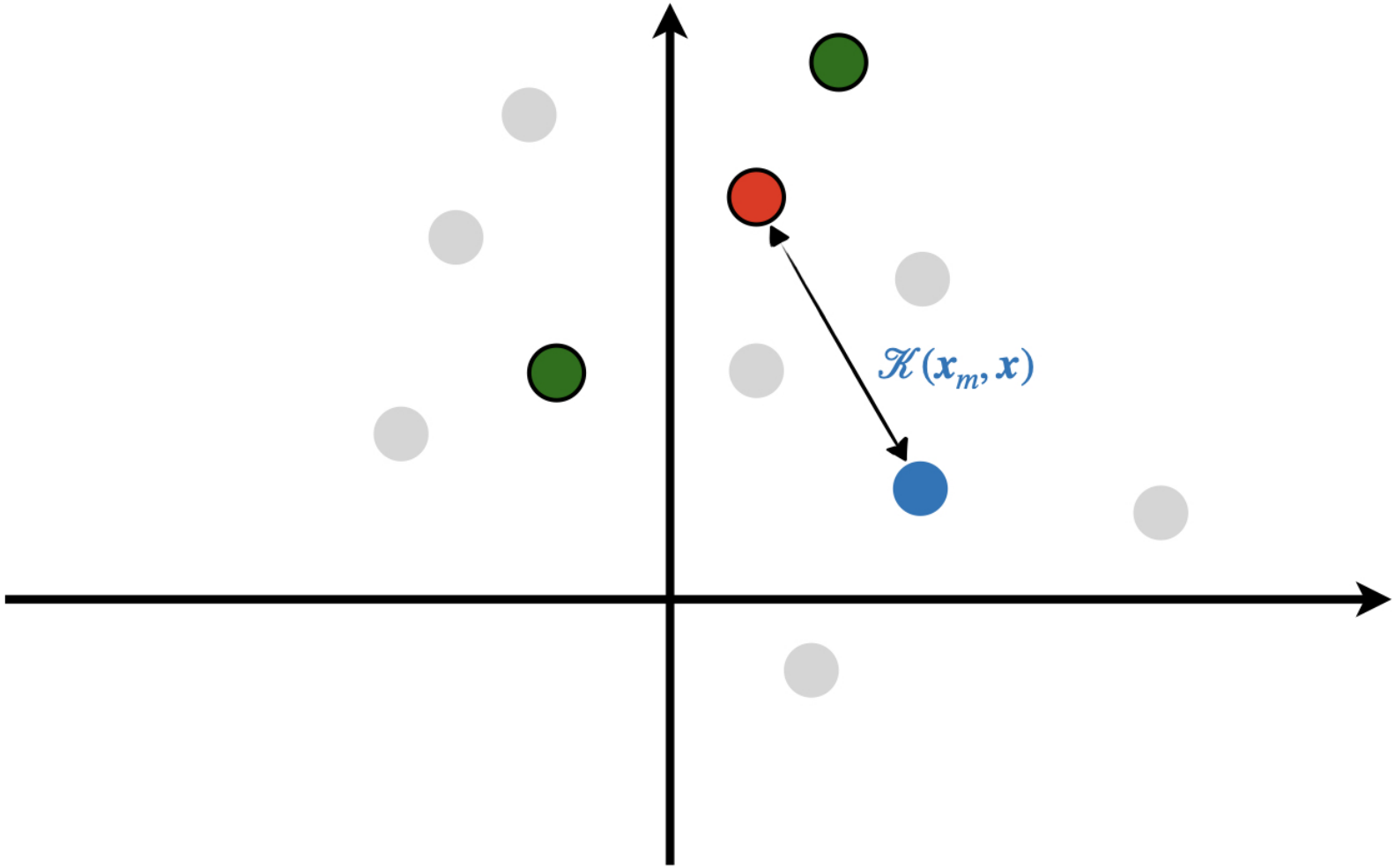
Support Vector Machines: *Visualization*



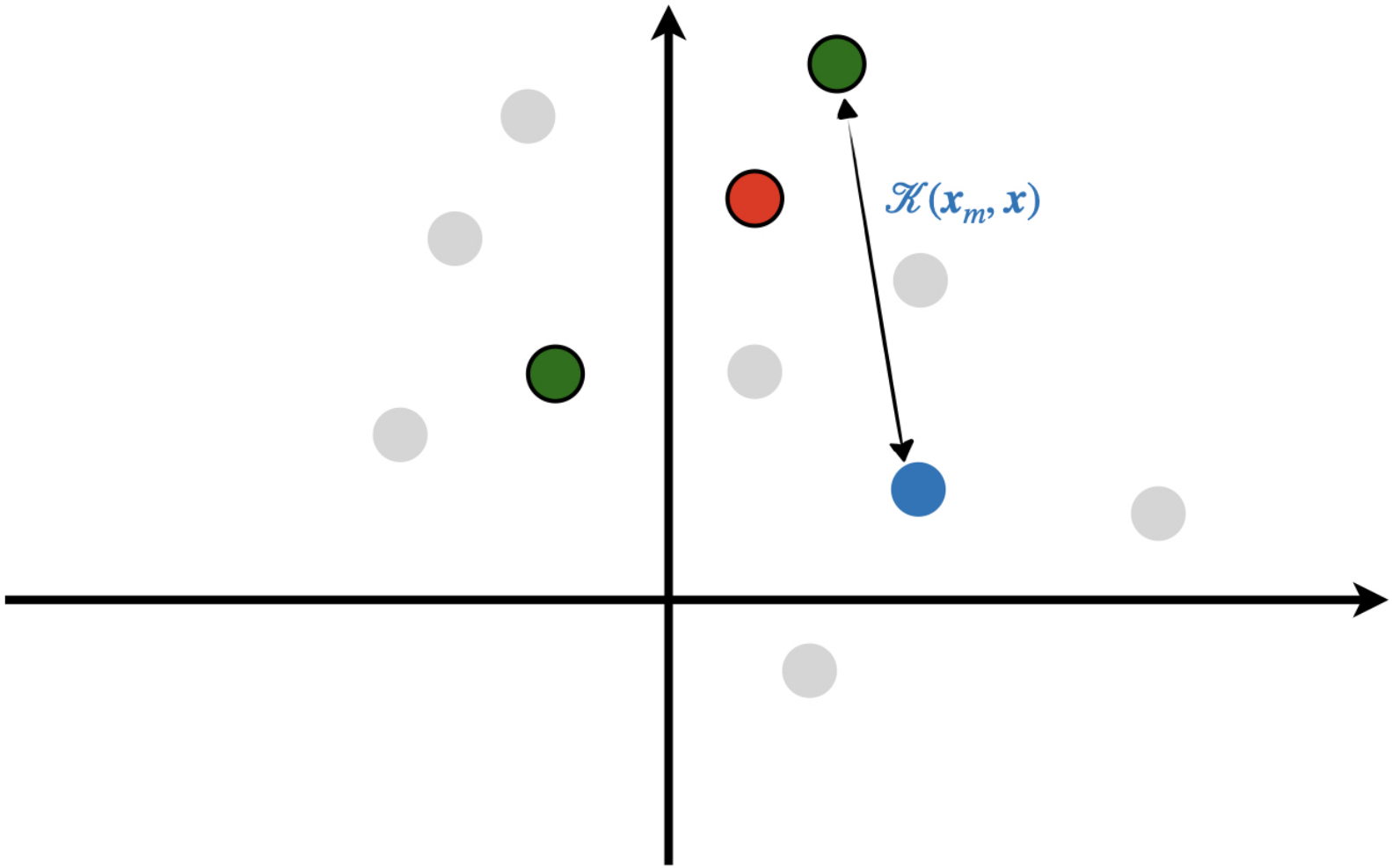
Support Vector Machines: *Visualization*



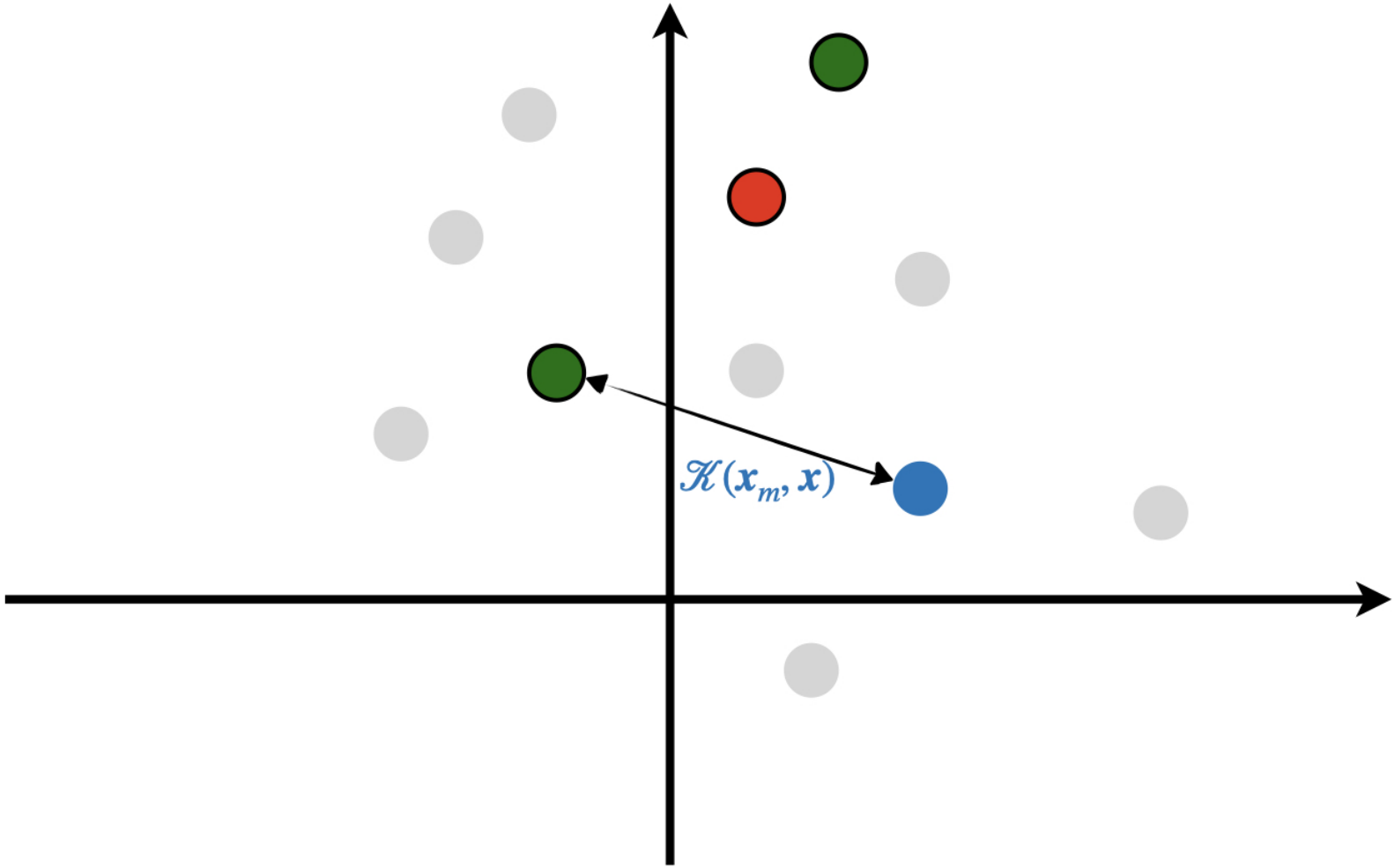
Support Vector Machines: *Visualization*



Support Vector Machines: *Visualization*



Support Vector Machines: *Visualization*



Famous Kernels: *Polynomial*

? *What should we choose as the kernel?*

! *There are some known choices*

Polynomial Kernel

The polynomial kernel of order p is defined as

$$\mathcal{K}\left(\boldsymbol{x}_n, \boldsymbol{x}_m\right) = \left(\boldsymbol{x}_n^\mathsf{T} \boldsymbol{x}_m + c\right)^p$$

Corresponding embedding computes polynomial features

Famous Kernels: *Polynomial – Example*

Say order is 2, the samples are scalars and set $c = 1$

$$\begin{aligned}\mathcal{K}(x_n, x_m) &= (x_n x_m + 1)^2 \\ &= x_n^2 x_m^2 + 2x_n x_m + 1 \\ &= \begin{bmatrix} x_m^2 & \sqrt{2}x_m & 1 \end{bmatrix} \begin{bmatrix} x_n^2 \\ \sqrt{2}x_n \\ 1 \end{bmatrix}\end{aligned}$$

This is like our Cat or Dog example

$$\varphi(x) = \begin{bmatrix} x^2 \\ \sqrt{2}x \\ 1 \end{bmatrix}$$

Famous Kernels: *Gaussian Kernel*

Gaussian (Radial) Kernel

The polynomial kernel of order p is defined as

$$\mathcal{K}\left(\boldsymbol{x}_n, \boldsymbol{x}_m\right) = \exp\left\{-\frac{\|\boldsymbol{x}_n - \boldsymbol{x}_m{}^2\|}{\sigma}\right\}$$

The Gaussian kernel corresponds to

embedding in infinite-dimensional space!

Try to use Taylor series expansion of exponential function to see this

$$\exp\{x\} = 1 + x + \frac{x^2}{2!} + \cdots = \sum_{i=0}^{\infty} \frac{x^i}{i!}$$

Preset Kernel ≡ *Feature Engineering*

? *How do we know if we have chosen a good kernel?*

! *Nobody knows!*

Feature Engineering

By choosing a predefined kernel, we are implicitly setting the embedding functions. This is often called feature engineering, since we indirectly use a fixed rule for extracting features

? *But is there any other way?*

! *Maybe, we can "let data speaks by itself"!*

Representation Learning: *Learning Kernels*

We can instead set the kernel to be a parameterized function

$$\mathcal{K}_{\boldsymbol{\theta}}\left(\boldsymbol{x}_m, \boldsymbol{x}_n\right)$$

If we use the SVM, our final loss depends on both $\mathbf{w}$ and $\boldsymbol{\theta}$: *we can train both*

$$\min_{\boldsymbol{\theta}, \mathbf{w}} \hat{R}\left(\boldsymbol{\theta}, \mathbf{w}\right)$$

Representation Learning

We learn both the kernel and classifier together: this is like learning how to represent data first and then classify it

Example: *We can leave σ in Gaussian kernel undecided and find it jointly with $\mathbf{w}$*

How to Find Excellent Representation

? *How can we find the right form for the kernel?*

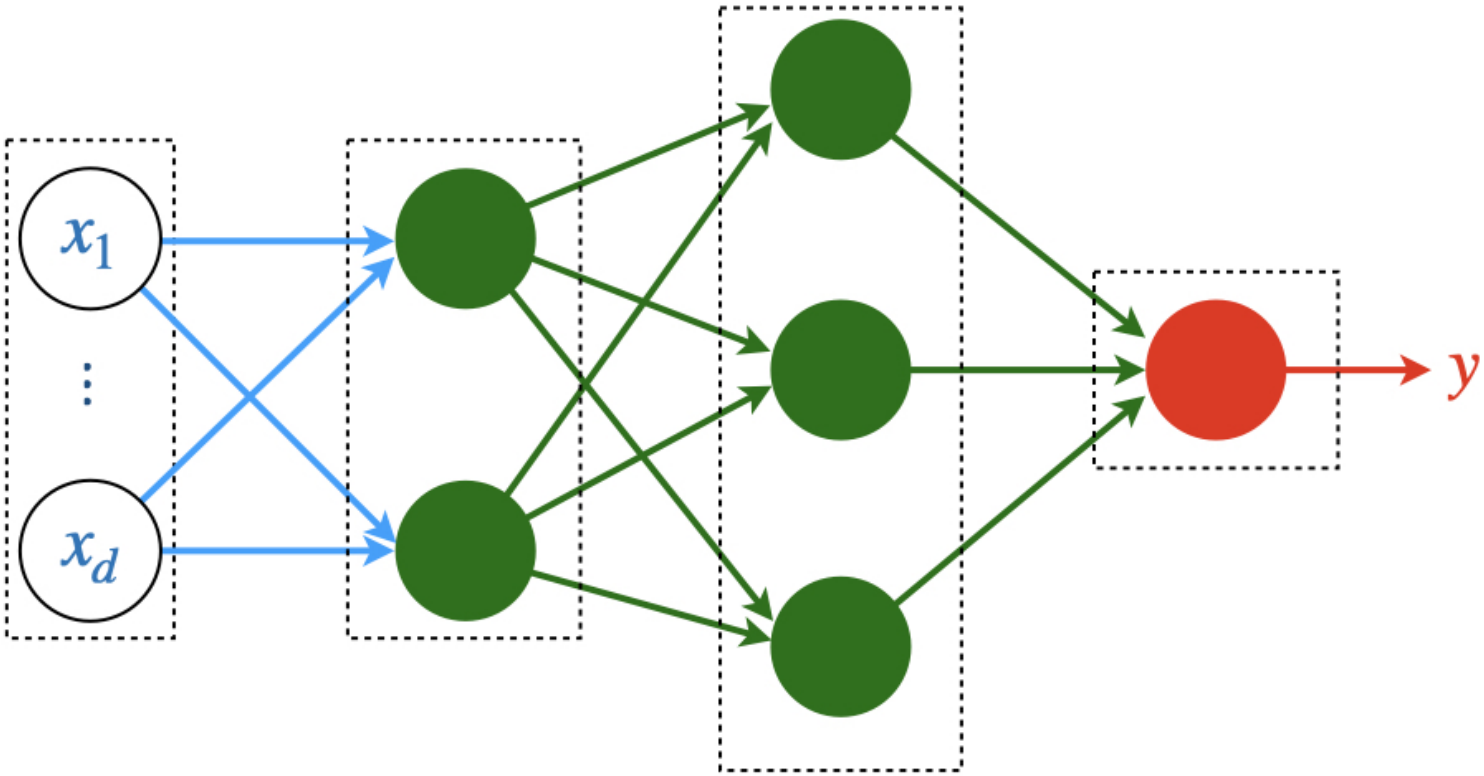
There is a rather rich literature on it

This is why it has its own name: Representation Learning

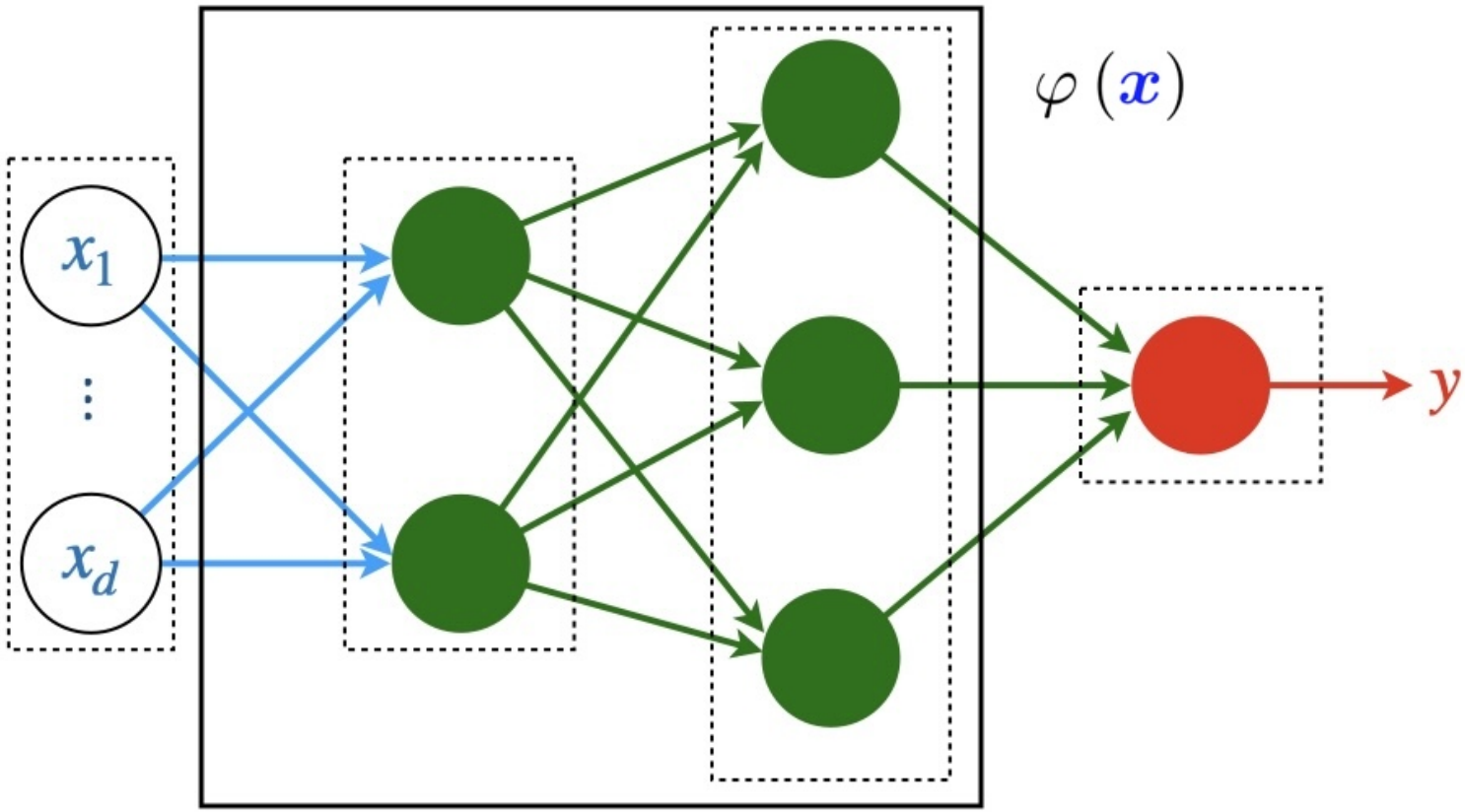
But it later turned out that

By repeating the linear model over and over we can build excellent kernels

Deep Learning vs Representation Learning



Deep Learning vs Representation Learning



Observation: *Neural Networks Give Excellent Representation*

? *How can we find the right form for the kernel?*

This resembles what we know as *neural networks*:

deep neural networks can make us great kernels!

NNs are hence what we are going to study next!

Further Read

- Bishop
  - ↳ Chapter 6: *Sections 6.2 – 6.4* ***SVM***
- ESL
  - ↳ Chapter 12: *Section 12.3 – 12.4* ***SVM***